

Autoregressive Process

Course: Practical Time Series Analysis

Lecturer: Dr. David Zmiaikou

April 12, 2024

Contents

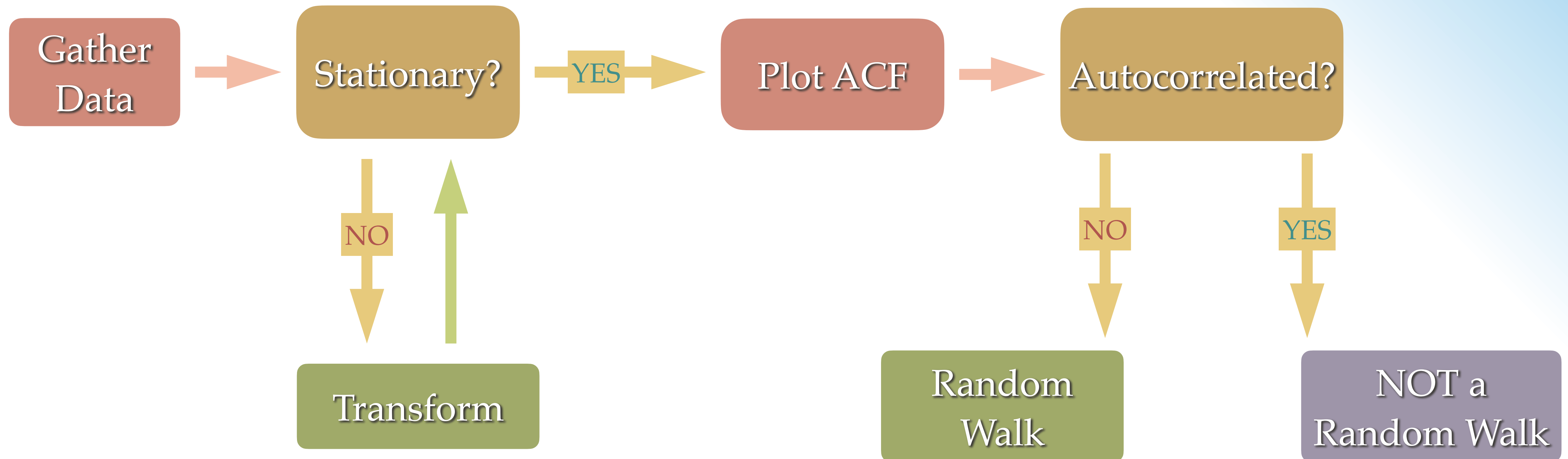
- 01 Forecasting a Moving Average Process**

- 02 Autoregressive Process**

- 03 Implementation**

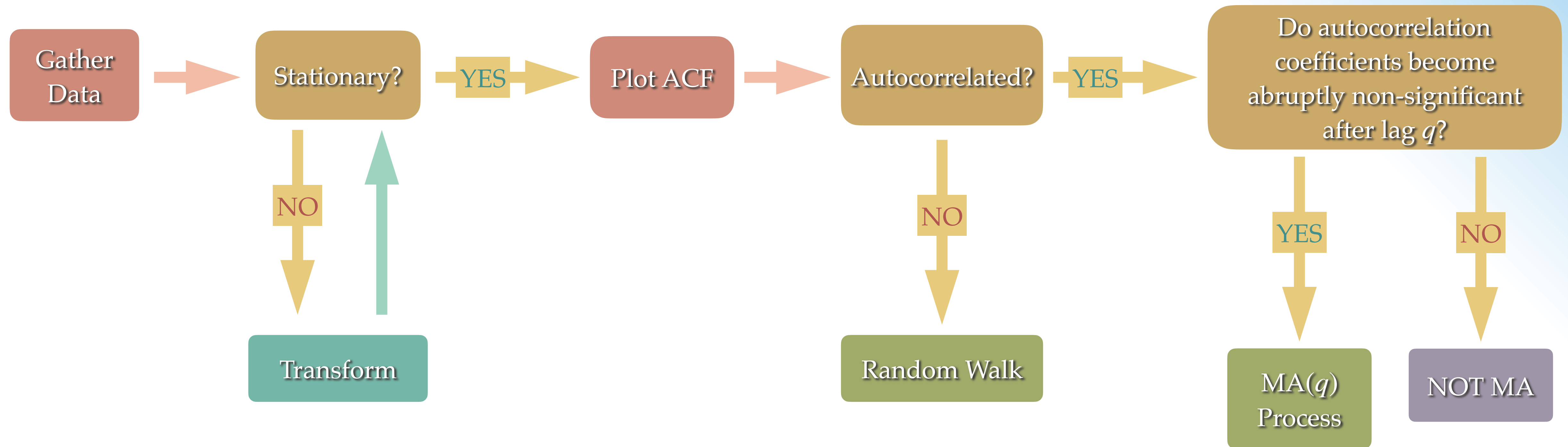
- 04 Partial Autocorrelation Function (PACF)**

Identifying a Random Walk



In the context of time series, **a random walk** is defined as a series whose first difference is *stationary* and *uncorrelated*.

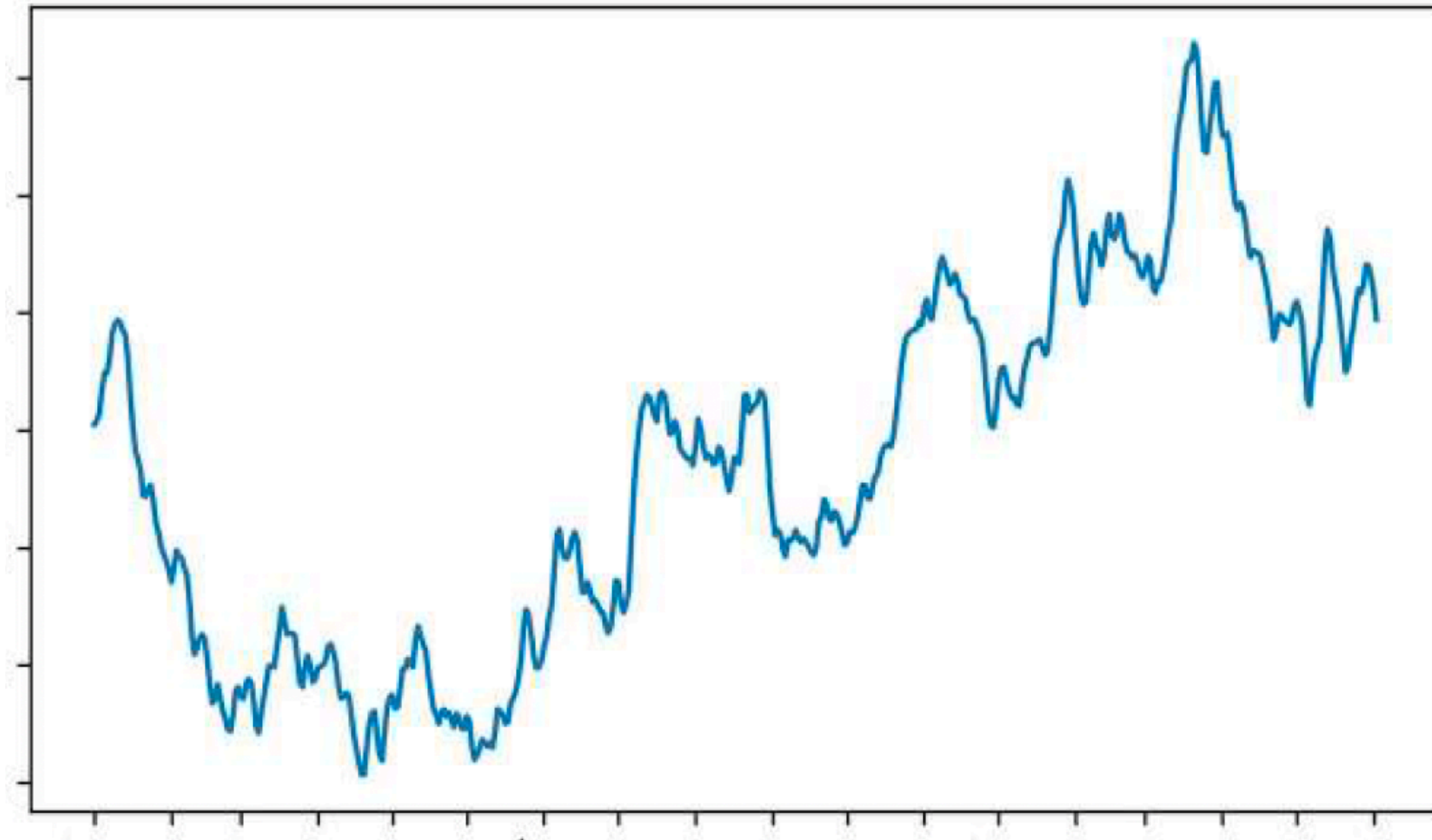
Identifying a Moving Average Process



01

Forecasting a Moving Average Process

^{def} What is a Moving Average Process?

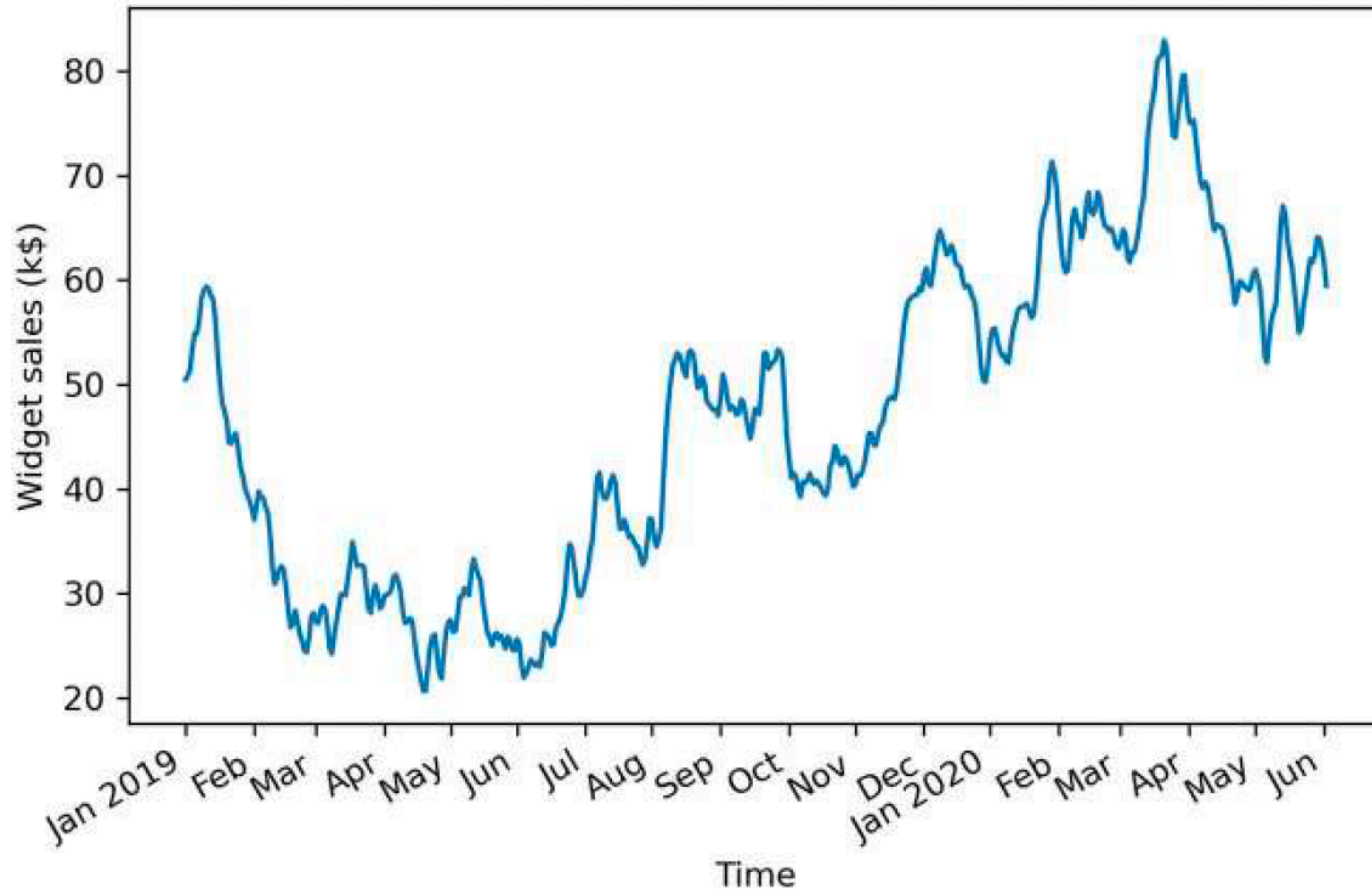


- In a **moving average (MA) process**, the current value depends linearly on the mean of the series, the current error term, and past error terms.
- The moving average model is denoted as $MA(q)$, where q is the **order**. The general expression of an $MA(q)$ model is

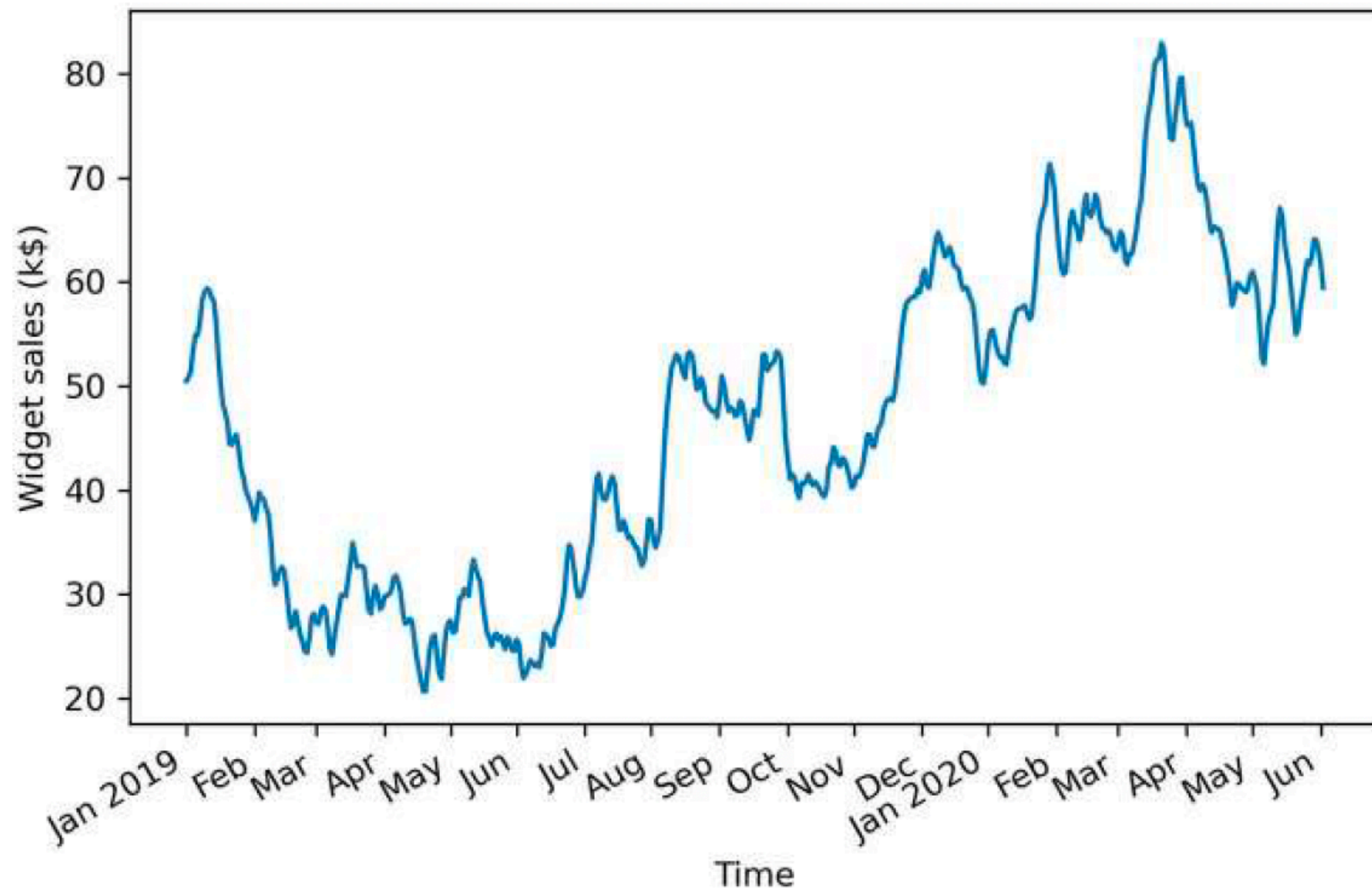
$$y_t = \mu + \epsilon_t + \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2} + \cdots + \theta_q \epsilon_{t-q}.$$

- The error terms are assumed to be mutually *independent* and *normally distributed*, just like white noise.
- The magnitude of the impact of past errors on the present value is quantified using a coefficient.

Example 1: Widget sales for the XYZ Widget Company over 500 days

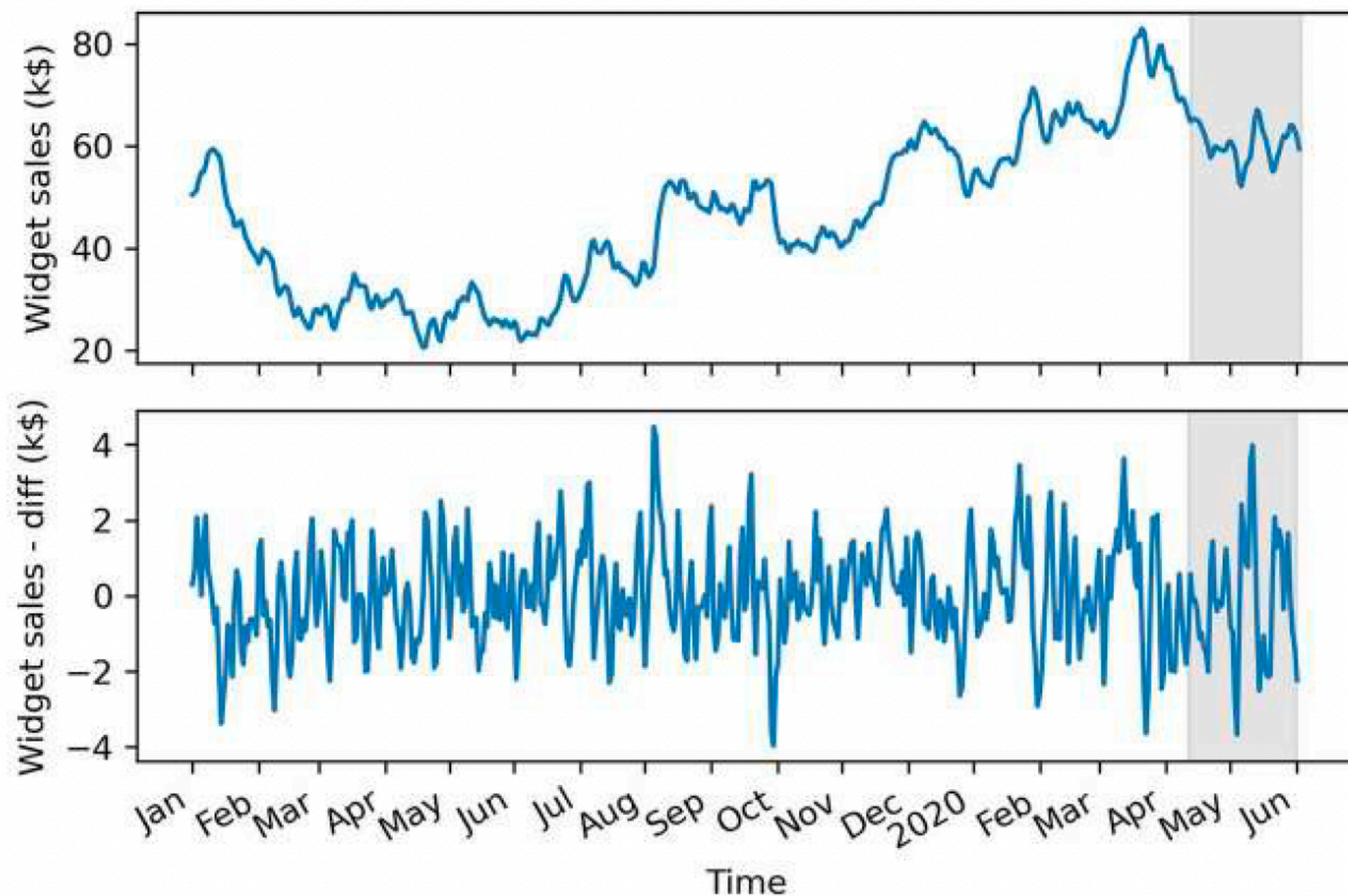


Forecasting a Moving Average Process



- The moving average model assumes stationarity, meaning that our forecasts must be done on a stationary time series.
- We will try two naive forecasting techniques and fit a second-order moving average model.
- The naive forecasts will serve as *baselines* to evaluate the performance of the moving average model, which we expect to be better than the baselines, since we previously identified our process to be a moving average process of order 2.
- Once we obtain our forecasts for the stationary process, we will have to **inverse-transform the forecasts**, meaning that we must undo the process of differencing to bring the forecasts back to their original scale.

Forecasting a Moving Average Process



- We will allocate 90% of the data to the train set and reserve the other 10% for the test set (we must forecast 50 timesteps into the future):

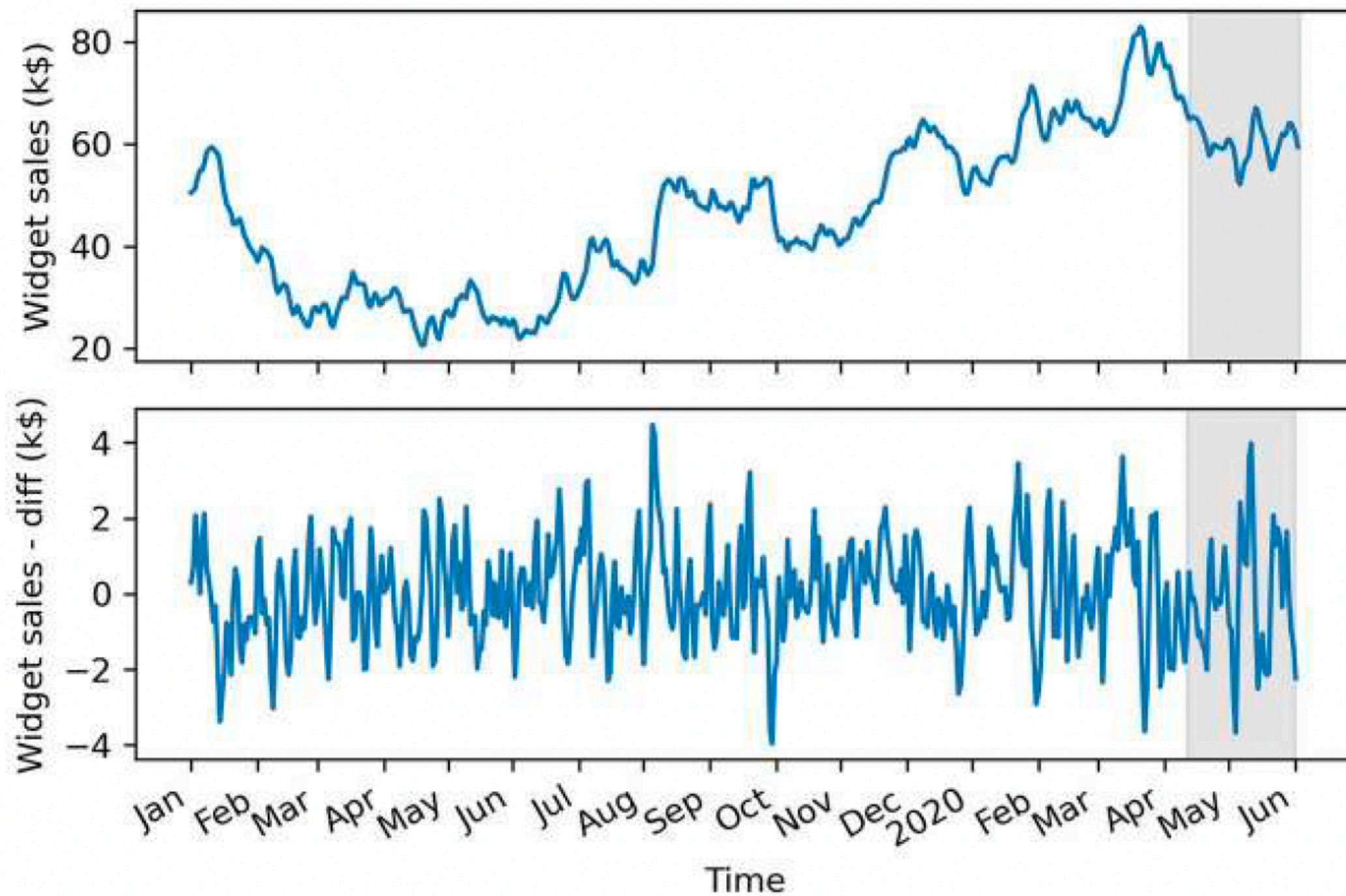
```
import pandas as pd
```

```
df_diff = pd.DataFrame({'widget_sales_diff':  
widget_sales_diff})
```

```
train = df_diff[:int(0.9*len(df_diff))]
```

```
test = df_diff[int(0.9*len(df_diff)):]
```

Forecasting with the MA(q) Model

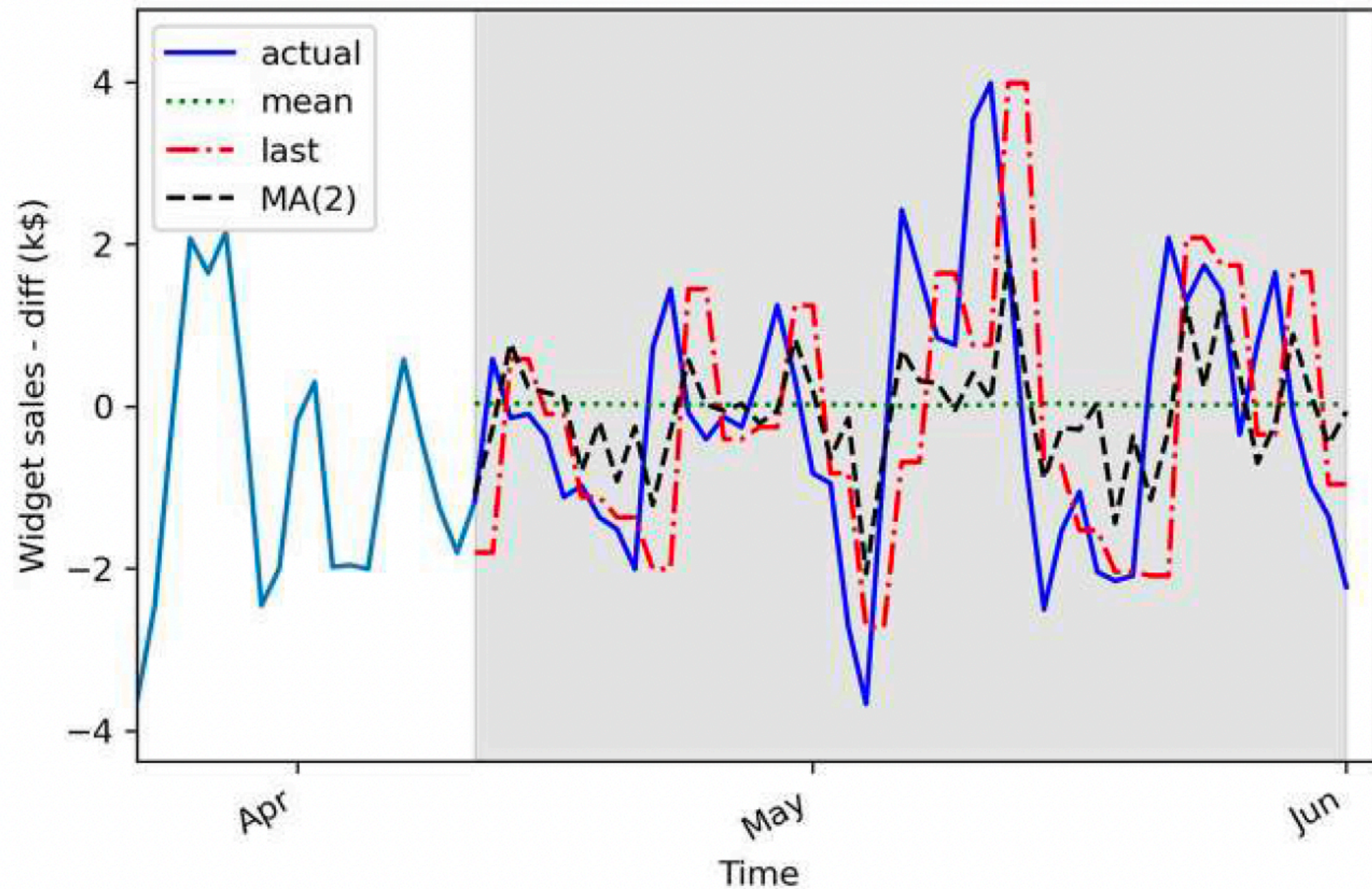


- When using an MA(q) model, forecasting beyond steps into the future will simply return the mean, because there are no error terms to estimate beyond q steps.
- We can use rolling forecasts to predict up to steps at a time in order avoid predicting only the mean of the series:

def rolling_forecast(df: pd.DataFrame, train_len: int, horizon: int, window: int, method: str) -> list:

- Parameters:
 - **train_len** is the number of data points that can be used to fit a model;
 - **horizon** is the length of the test set and represents how many values must be predicted;
 - **window** specifies how many timesteps are predicted at a time (for an MA(2) process, window is be equal to 2.);
 - **method** specifies what model to use.

Forecasting a Moving Average Process: Historical Mean



- Each forecasting method is run in a loop. The loop starts at the end of the training set and continues until `total_len`, exclusive, with steps of `window` (`total_len` is the sum of `train_len` and `horizon`):

`total_len = train_len + horizon`

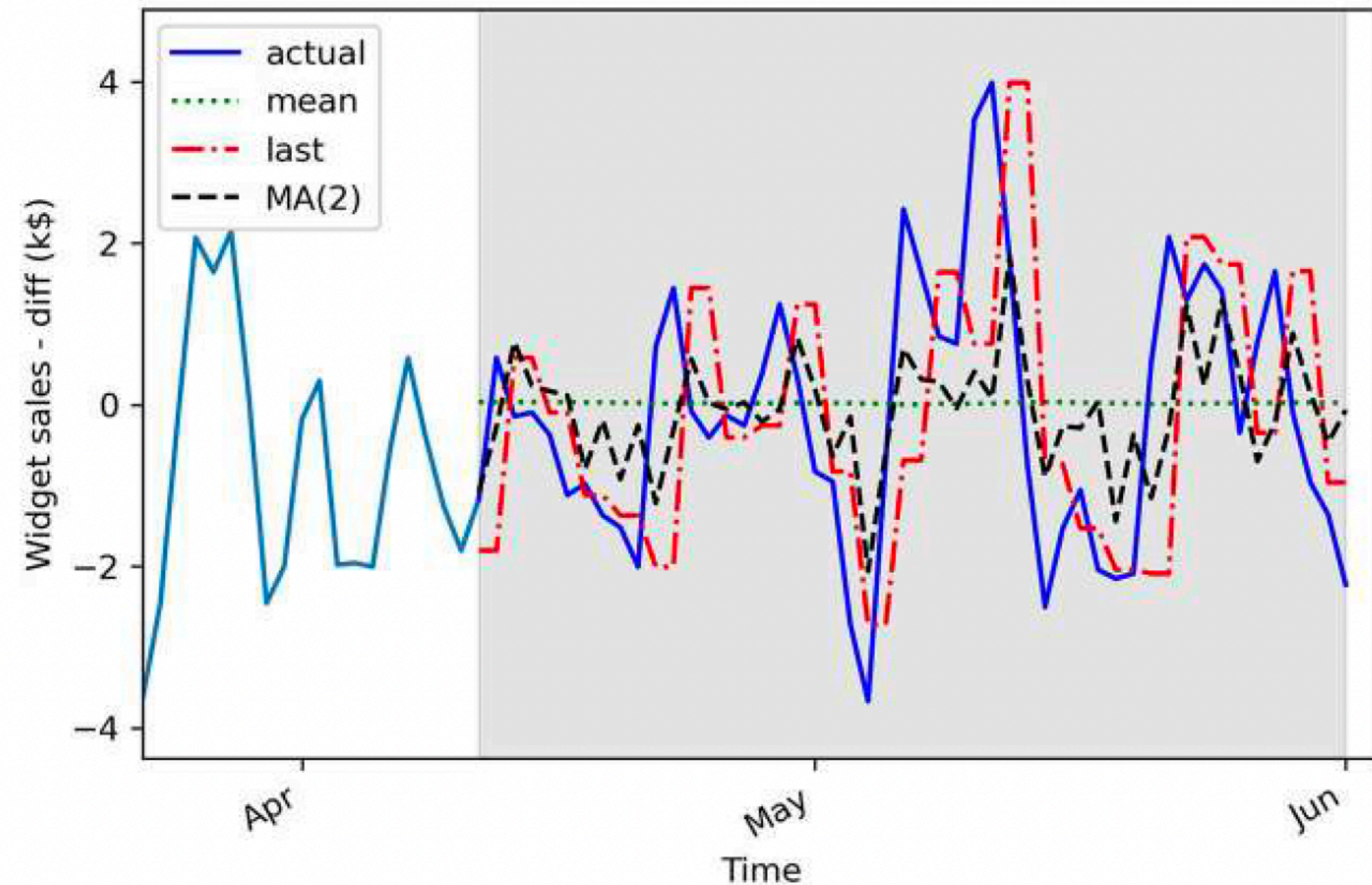
`if method == 'mean':`
`pred_mean = []`

`for i in range(train_len, total_len, window):`
`mean = np.mean(df[:i].values)`
`pred_mean.extend(mean for _ in range(window))`

`return pred_mean`

- The loop generates a list of 25 values, but each pass generates two forecasts, thus returning a list of 50 forecasts for the entire test set.

Forecasting a Moving Average Process: Last Known Value



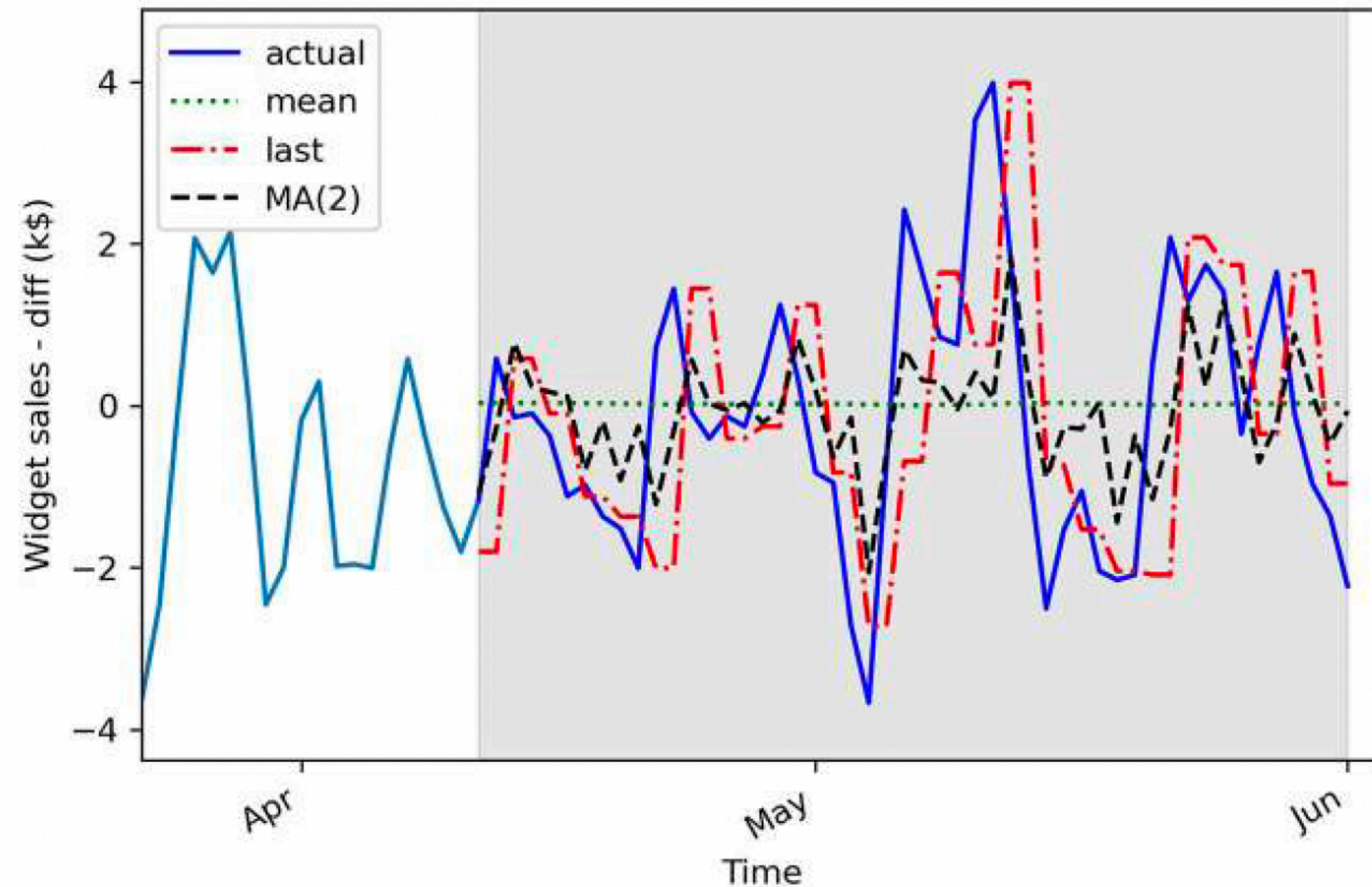
- Another possible baseline is to predict the last known value of the train set:

```
elif method == 'last':  
    pred_last_value = []
```

```
for i in range(train_len, total_len, window):  
    last_value = df[:i].iloc[-1].values[0]  
    pred_last_value.extend(last_value for _  
                           in range(window))
```

```
return pred_last_value
```

Forecasting a Moving Average Process: MA(2)



- Import the **SARIMAX** function from the statsmodels library:

```
from statsmodels.tsa.statespace.sarimax import SARIMAX
```

- This function fits an MA(2) model to our differenced series. Note that SARIMAX is a complex model that allows to consider seasonal effects, autoregressive processes, non-stationary time series, moving average processes, and exogenous variables all in a single model.

```
elif method == 'MA':
```

```
    pred_MA = []
```

```
    for i in range(train_len, total_len, window):
```

```
        model = SARIMAX(df[:i], order=(0,0,2))
```

```
        res = model.fit(dis=False)
```

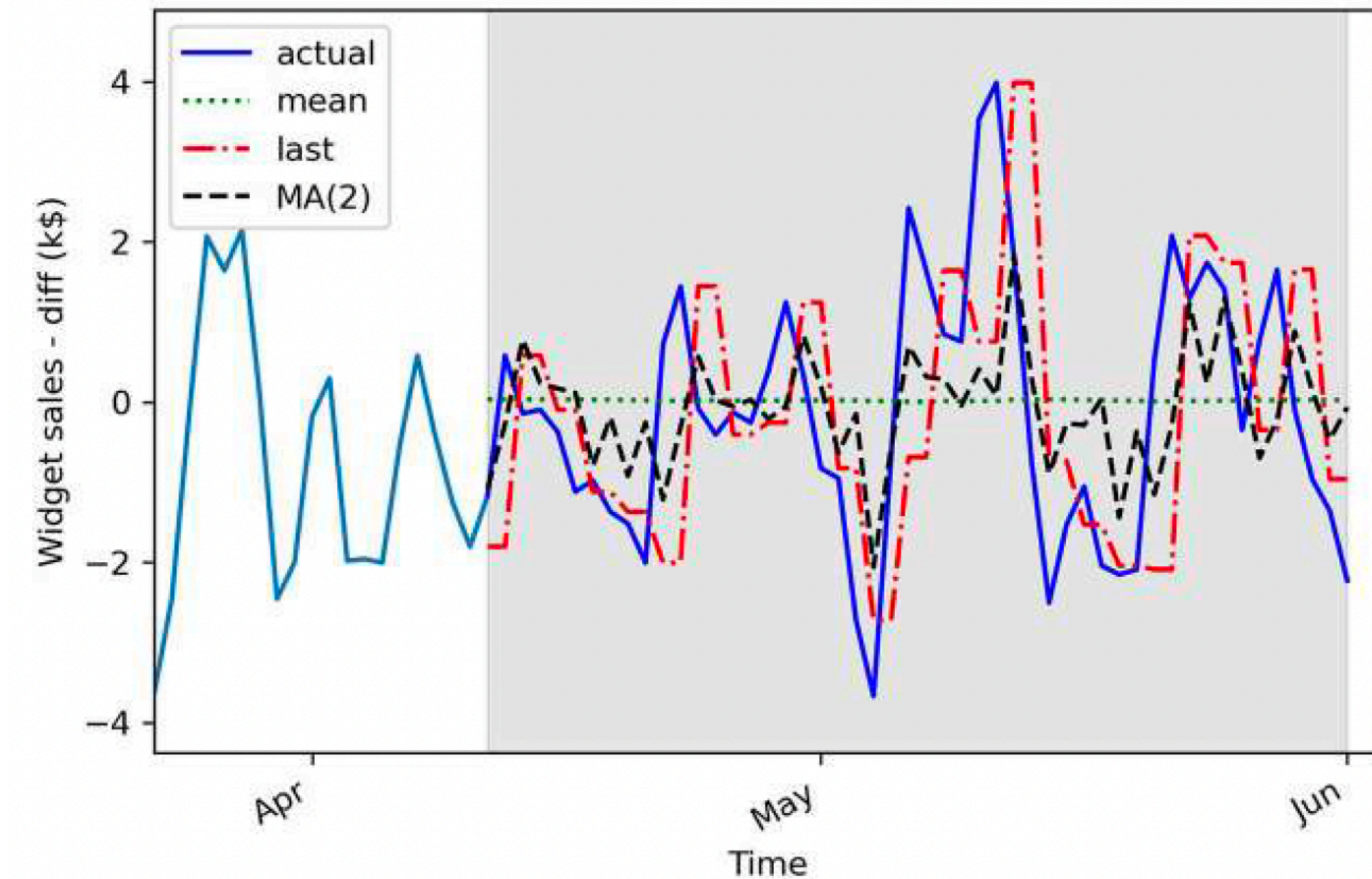
```
        predictions = res.get_prediction(0, i + window - 1)
```

```
        oos_pred = predictions.predicted_mean.iloc[-window:]
```

```
        pred_MA.extend(oos_pred)
```

```
    return pred_MA
```

Forecasting a Moving Average Process



- Use our `rolling_forecast` function to generate a list of predictions for each method. Each list of predictions is then stored in its own column in `pred_df`:

```
pred_df = test.copy()
```

```
TRAIN_LEN = len(train)
```

```
HORIZON = len(test)
```

```
WINDOW = 2
```

```
pred_mean = rolling_forecast(df_diff,  
                             TRAIN_LEN, HORIZON, WINDOW, 'mean')
```

```
pred_last_value = rolling_forecast(df_diff,  
                                    TRAIN_LEN, HORIZON, WINDOW, 'last')
```

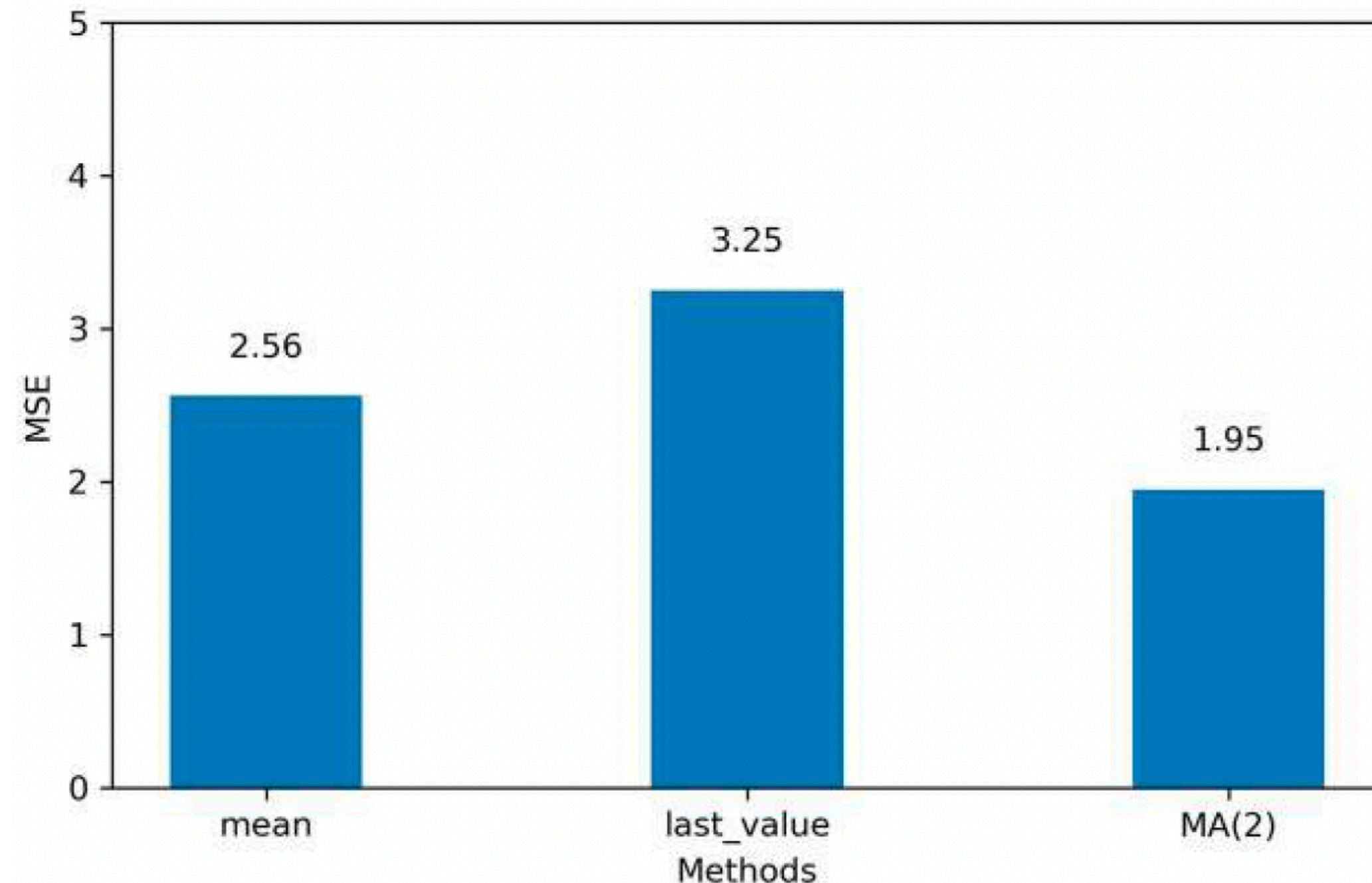
```
pred_MA = rolling_forecast(df_diff,  
                            TRAIN_LEN, HORIZON, WINDOW, 'MA')
```

```
pred_df['pred_mean'] = pred_mean
```

```
pred_df['pred_last_value'] = pred_last_value
```

```
pred_df['pred_MA'] = pred_MA
```

Forecasting a Moving Average Process (Comparison)



- Measure the MSE:

```
from sklearn.metrics import mean_squared_error
```

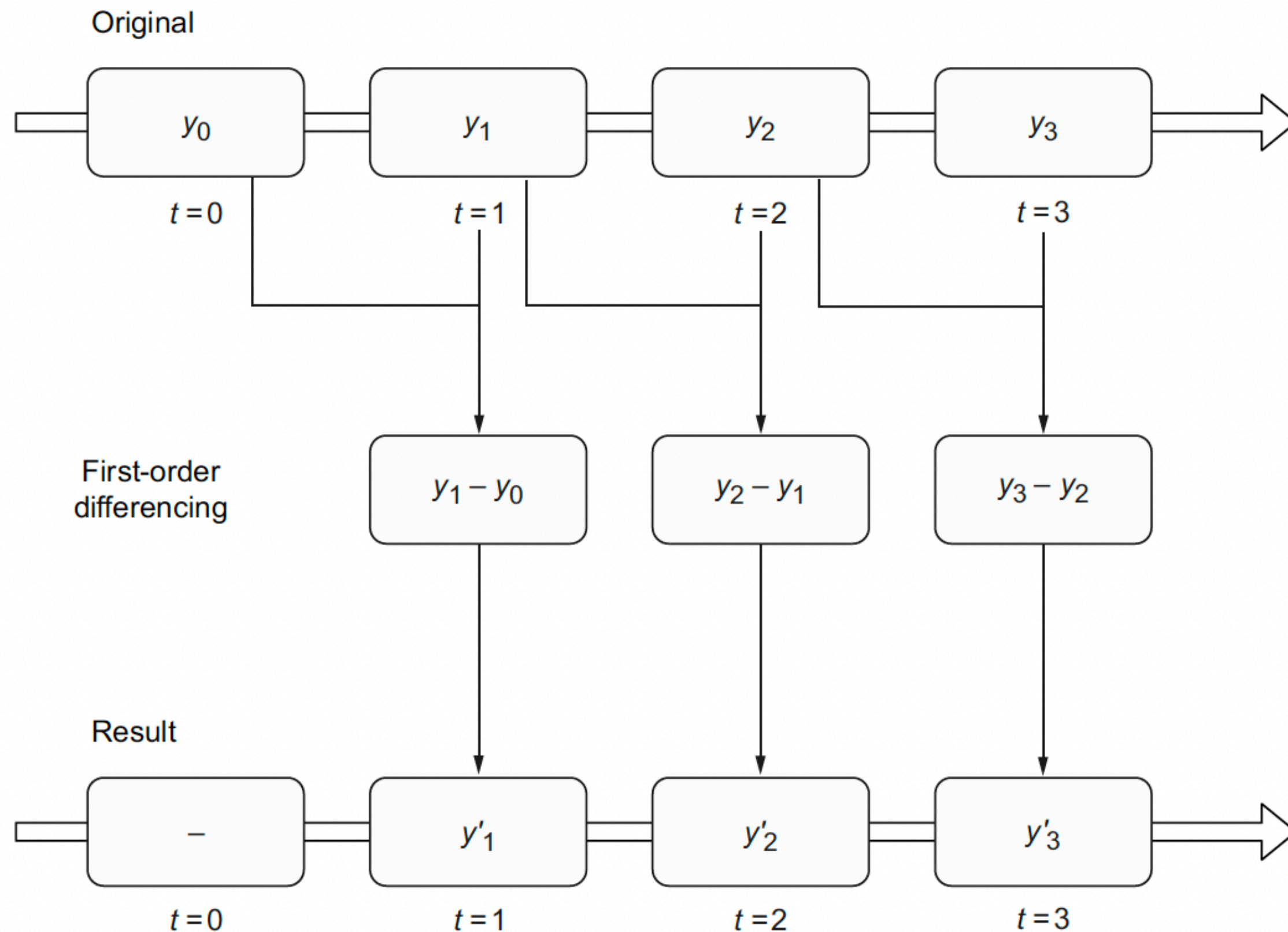
```
mse_mean = mean_squared_error(pred_df['widget_sales_diff'],  
pred_df['pred_mean'])
```

```
mse_last = mean_squared_error(pred_df['widget_sales_diff'],  
pred_df['pred_last_value'])
```

```
mse_MA = mean_squared_error(pred_df['widget_sales_diff'],  
pred_df['pred_MA'])
```

- This gives an MSE of 2.56 for the historical mean method, 3.25 for the last value method, and 1.95 for the MA(2) model.
- This is expected, because we previously identified a second-order moving average process for the differenced volume of widget sales,

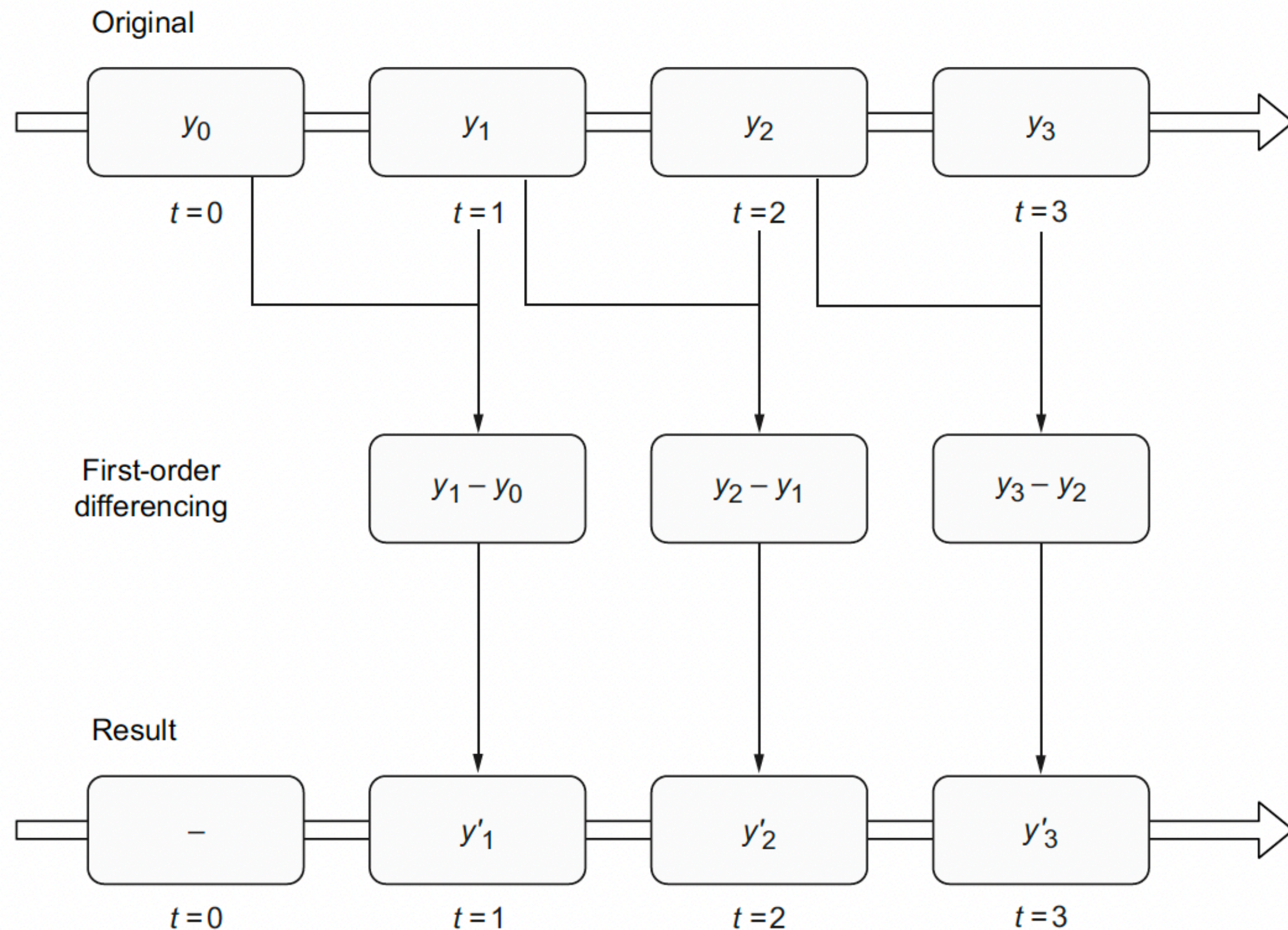
First-Order Differencing



- ❖ In order to make a time series stationary, we applied **differencing** (as many times as needed).
- ❖ **First-order differencing** calculates changes between neighbouring values:

$$y'_t = y_t - y_{t-1}$$

Reversing the First-Order Difference



❖ To recover the initial values y_t from the differenced values $y'_t = y_t - y_{t-1}$, we can use the relations:

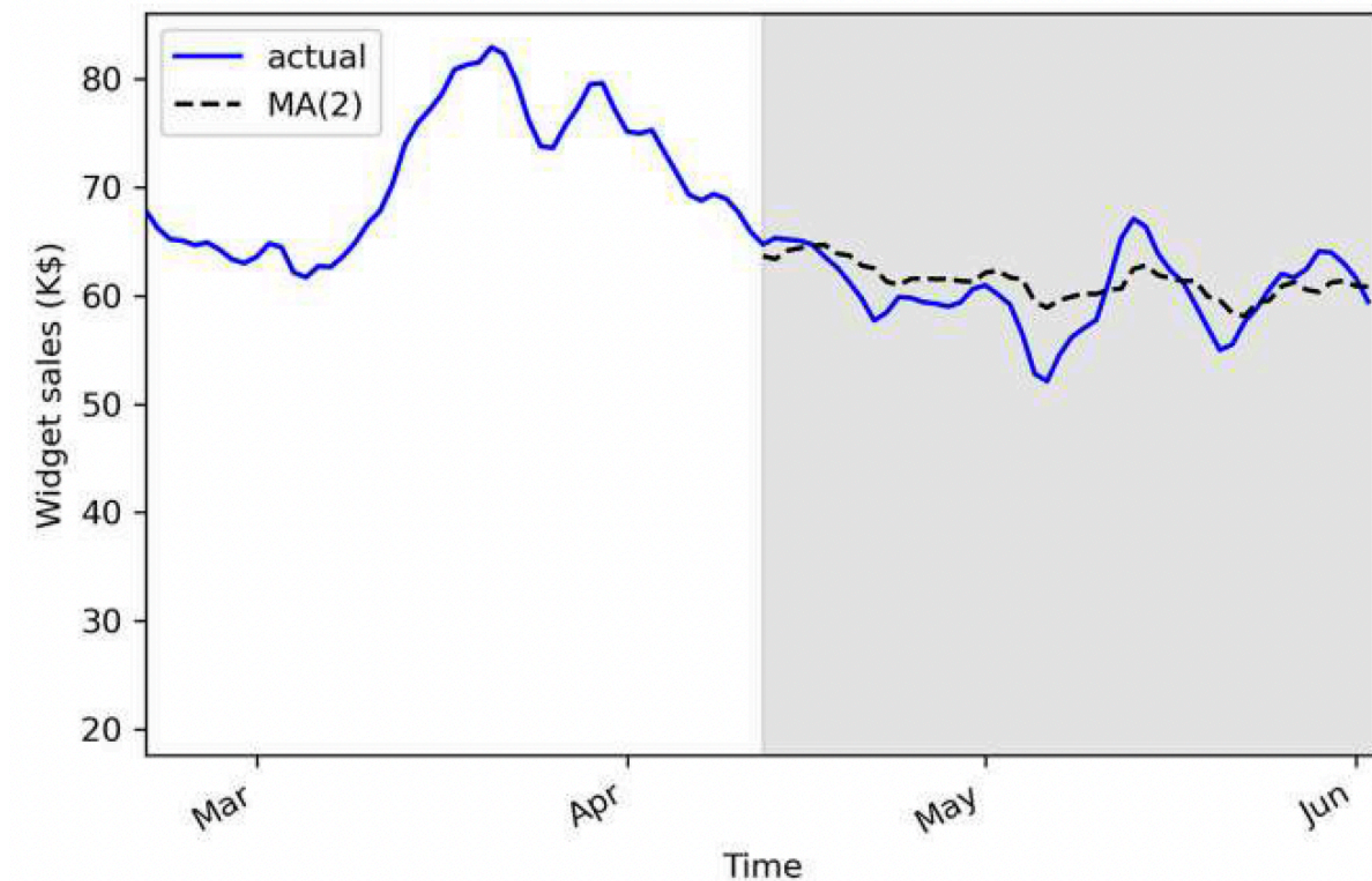
$$y_1 = y_0 + y'_1$$

$$y_2 = y_0 + y'_1 + y'_2$$

...

$$y_t = y_0 + y'_1 + y'_2 + \dots + y'_t$$

Forecasting a Moving Average Process

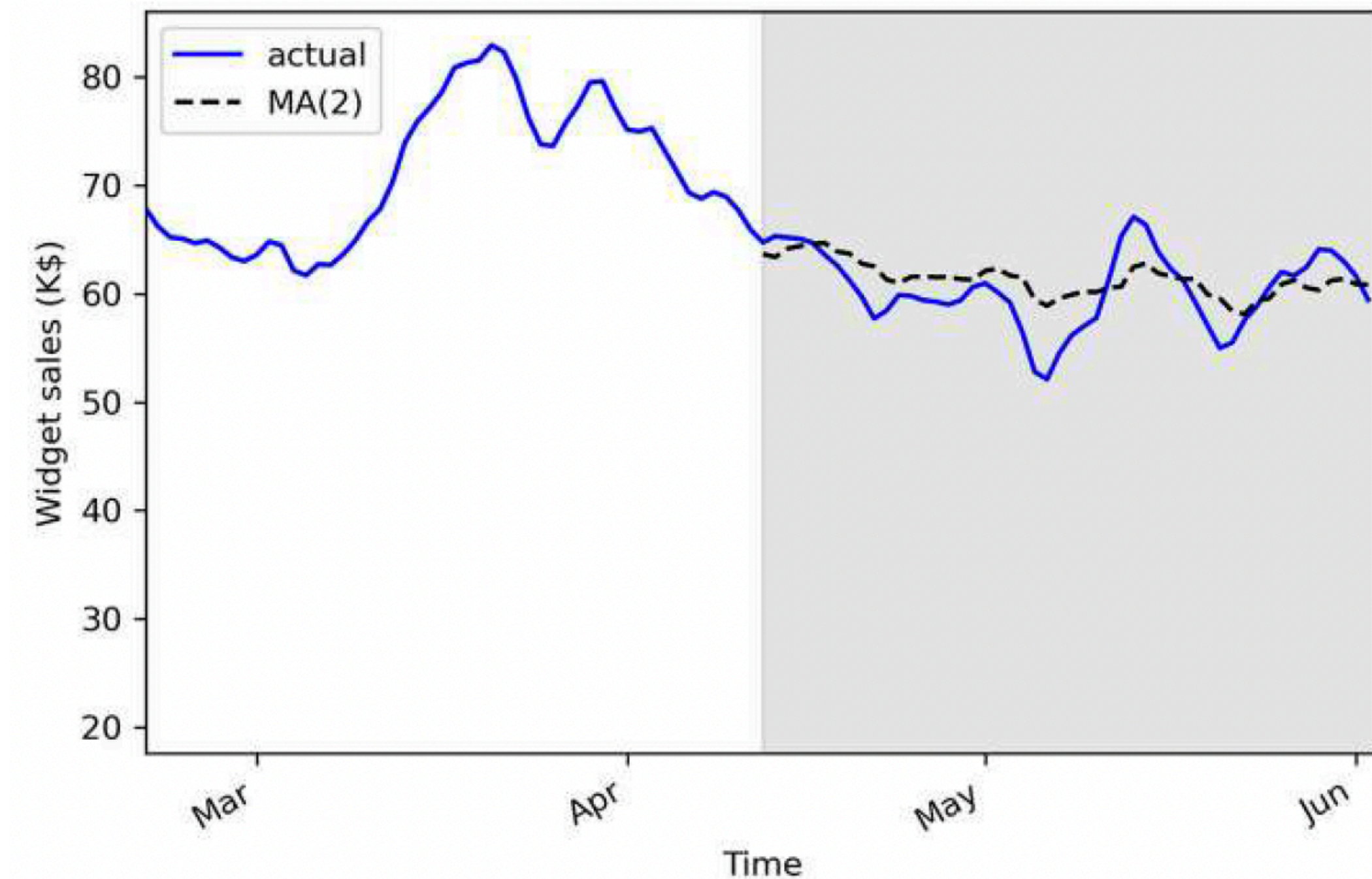


- Thus, to obtain our predictions in the original scale of our dataset, we need to use the first value of the test as our initial value. Then we can perform a cumulative sum to obtain a series of 50 predictions in the original scale of the dataset:

```
df['pred_widget_sales'] = pd.Series()
```

```
df['pred_widget_sales'][450:] =  
df['widget_sales'].iloc[450] + pred_df['pred_MA'].cumsum()
```

Forecasting a Moving Average Process



- We can measure the mean absolute error (MAE) using the **mean_absolute_error** function from **sklearn**. This metric is easy to interpret — it returns the average of the absolute difference between the predicted and actual values, instead of a squared difference like the MSE:

from sklearn.metrics import mean_absolute_error

***mae_MA_undiff =
mean_absolute_error(df['widget_sales'].iloc[450:],
df['pred_widget_sales'].iloc[450:])***

- This gives out an MAE of 2.32. Therefore, our predictions are on average off by \$2,320, either above or below the actual value.

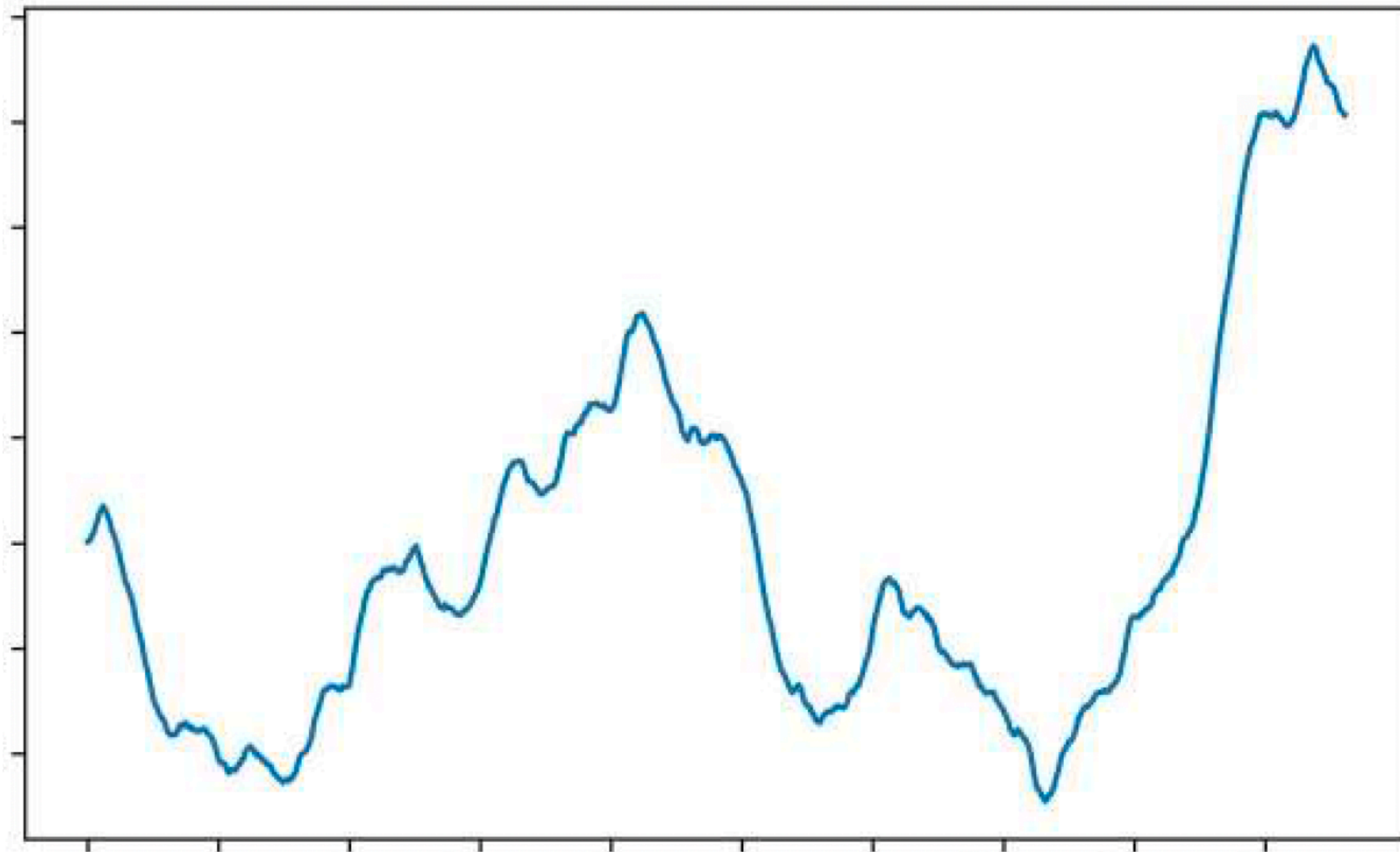
What is next?

- We covered the moving average process and how it can be modelled by an $MA(q)$ model, where q is the order. You learned that to identify a moving average process, you must study the ACF plot once it is stationary. The ACF plot will show significant peaks all the way to lag q , and the rest will not be significantly different from 0.
- However, it is possible that when studying the ACF plot of a stationary process, one sees a sinusoidal pattern, with negative coefficients and significant autocorrelation at large lags. In this case, it is *not* a moving average process.
- When we see a sinusoidal pattern in the ACF plot of a stationary process, this is a hint that an **autoregressive process** is at play, and we must use an $AR(p)$ model to produce our forecast.

02

Autoregressive Process

^{def} What is an Autoregressive Process?

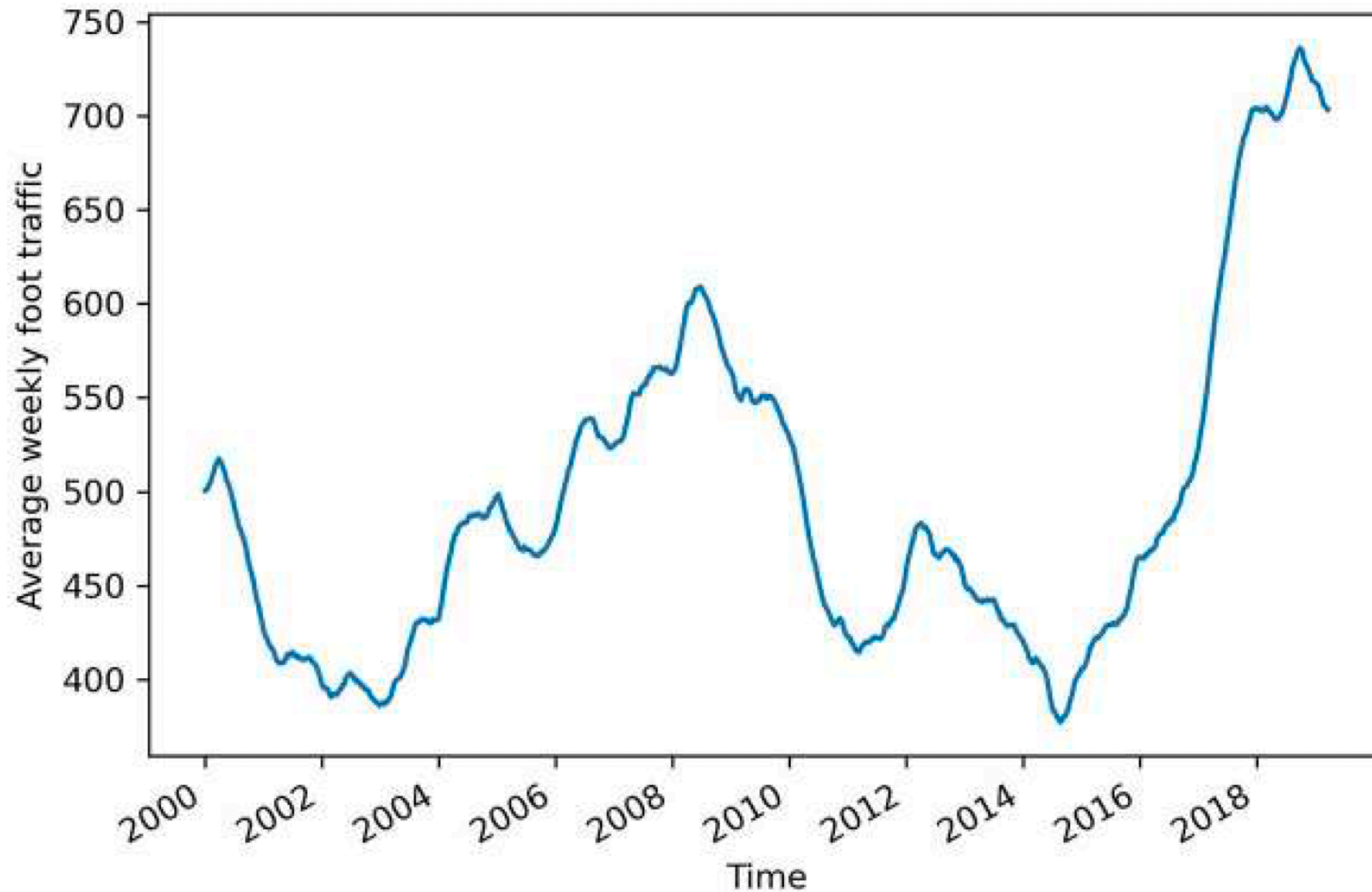


- An **autoregressive process** is a regression of a variable against itself. In a time series, this means that the present value is linearly dependent on its past values.
- The autoregressive process is denoted as $AR(p)$, where p is the **order**. The general expression of an $AR(p)$ model is

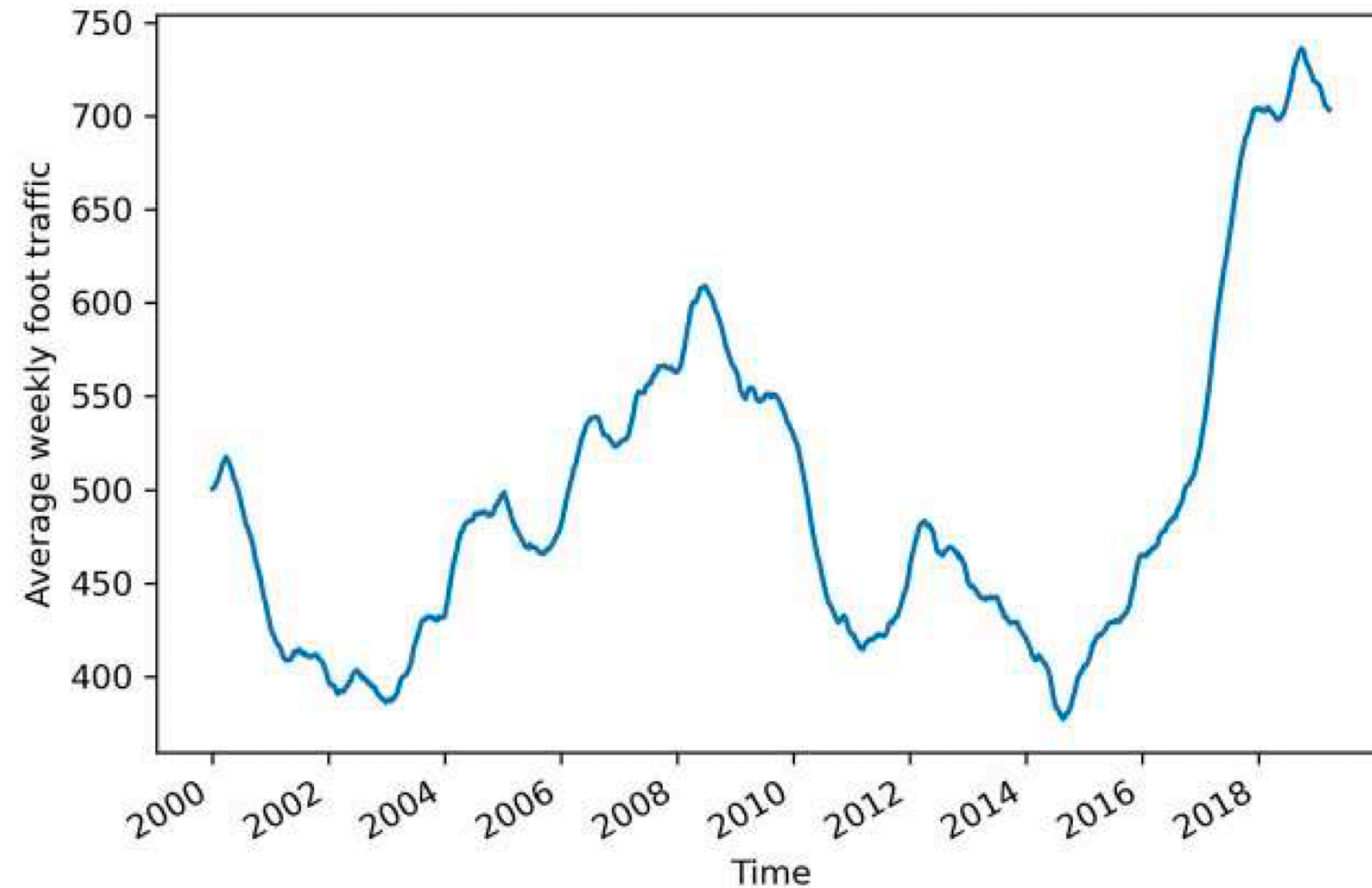
$$y_t = C + \epsilon_t + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \dots + \phi_p y_{t-p},$$

where C is a constant, ϵ_t is an error (white noise) and ϕ_t are coefficients (magnitude of influence of past values).

Example 2: Average weekly foot traffic at a retail store

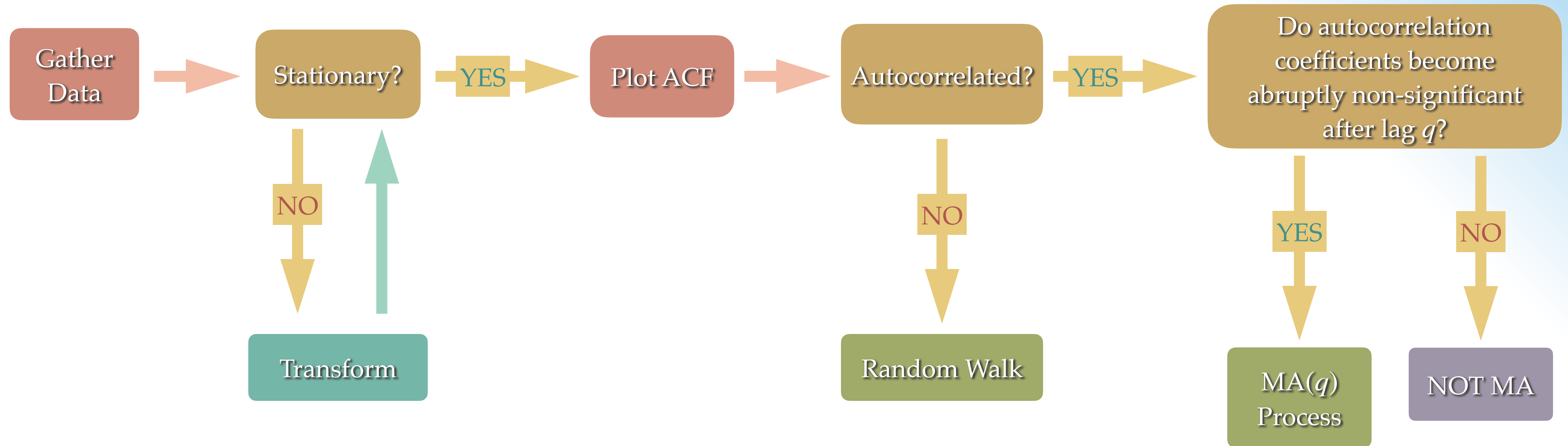


Example 2: Average weekly foot traffic at a retail store

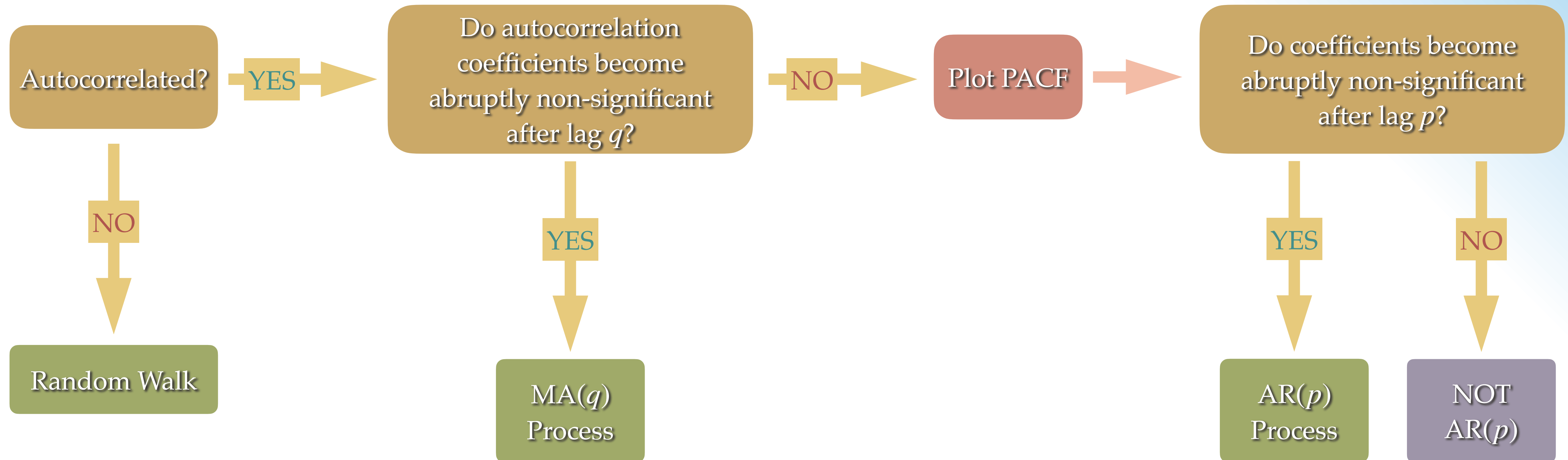


- Long-term trends with peaks and troughs.
- Not stationary.
- No apparent cyclical pattern.

Identifying an Autoregressive Process



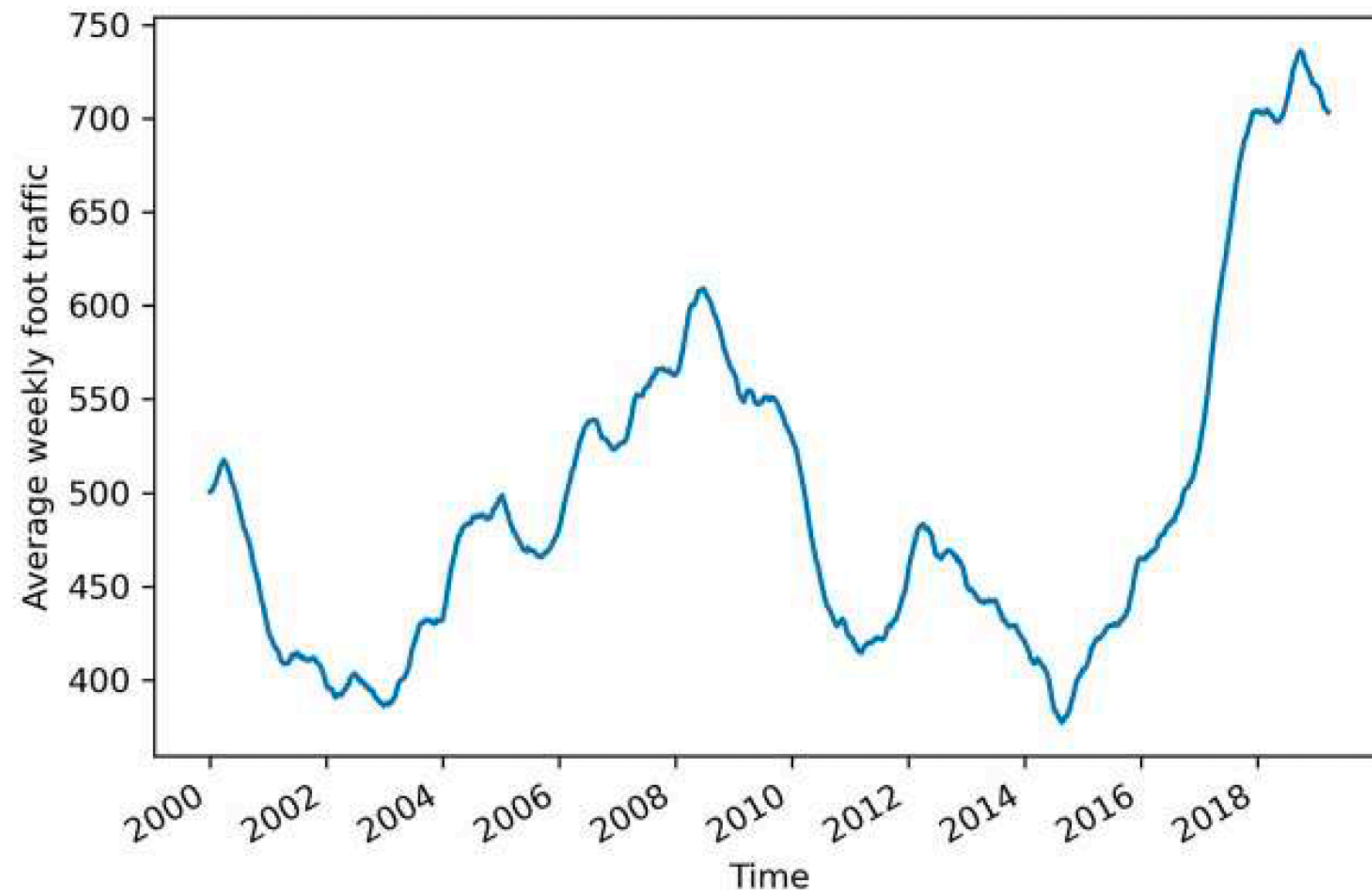
Identifying an Autoregressive Process



03

Implementation

Implementation



- We will use **DataFrame** from **pandas**:

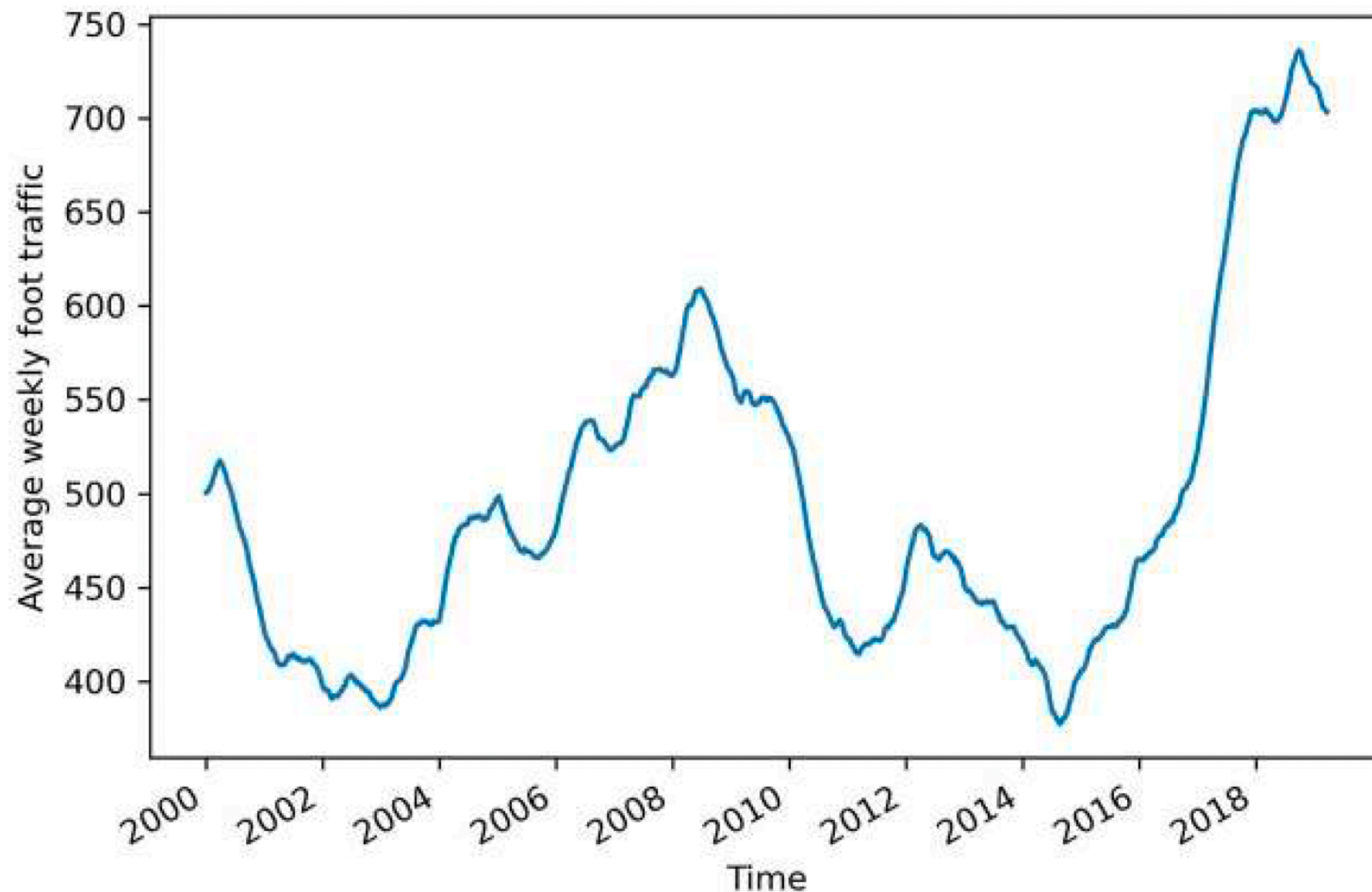
```
import pandas as pd  
df = pd.read_csv('foot_traffic.csv')
```

- Use the **adfuller** function from the **statsmodels** library and extract the ADF statistic and p-value. If the ADF statistic is a large negative number and the p-value is smaller than 0.05, our series is stationary. Otherwise, we must apply transformations.

```
from statsmodels.tsa.stattools import adfuller
```

```
ADF_result = adfuller(df['foot_traffic'])  
print(f'ADF Statistic: {ADF_result[0]}')  
print(f'p-value: {ADF_result[1]}')
```

Implementation (Stationarity)



- We will use **DataFrame** from **pandas**:

```
import pandas as pd  
df = pd.read_csv('foot_traffic.csv')
```

- Use the **adfuller** function from the **statsmodels** library and extract the ADF statistic and p-value. If the ADF statistic is a large negative number and the p-value is smaller than 0.05, our series is stationary. Otherwise, we must apply transformations.

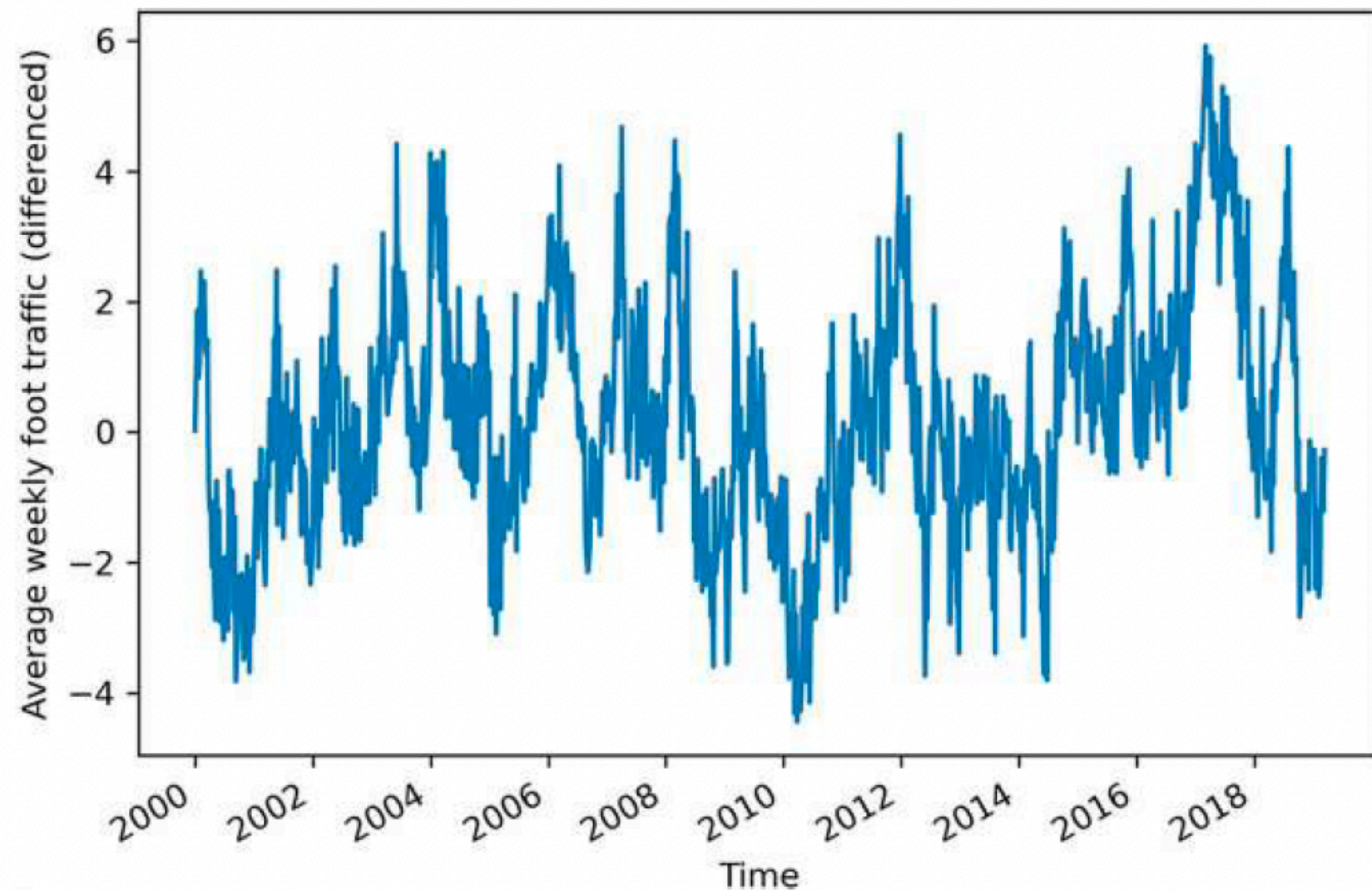
```
from statsmodels.tsa.stattools import adfuller
```

```
ADF_result = adfuller(df['foot_traffic'])  
print(f'ADF Statistic: {ADF_result[0]}')  
print(f'p-value: {ADF_result[1]}')
```

- This results in an ADF statistic of -1.18 and a p-value of 0.68 . Not stationary.
- Try to de-trend by applying a first-order differencing. Use the **diff** method from the **numpy** library:

```
import numpy as np  
foot_traffic_diff = np.diff(df['foot_traffic'], n=1)
```

Implementation (Stationarity)



- Try to de-trend by applying a first-order differencing. Use the **diff** method from the **numpy** library:

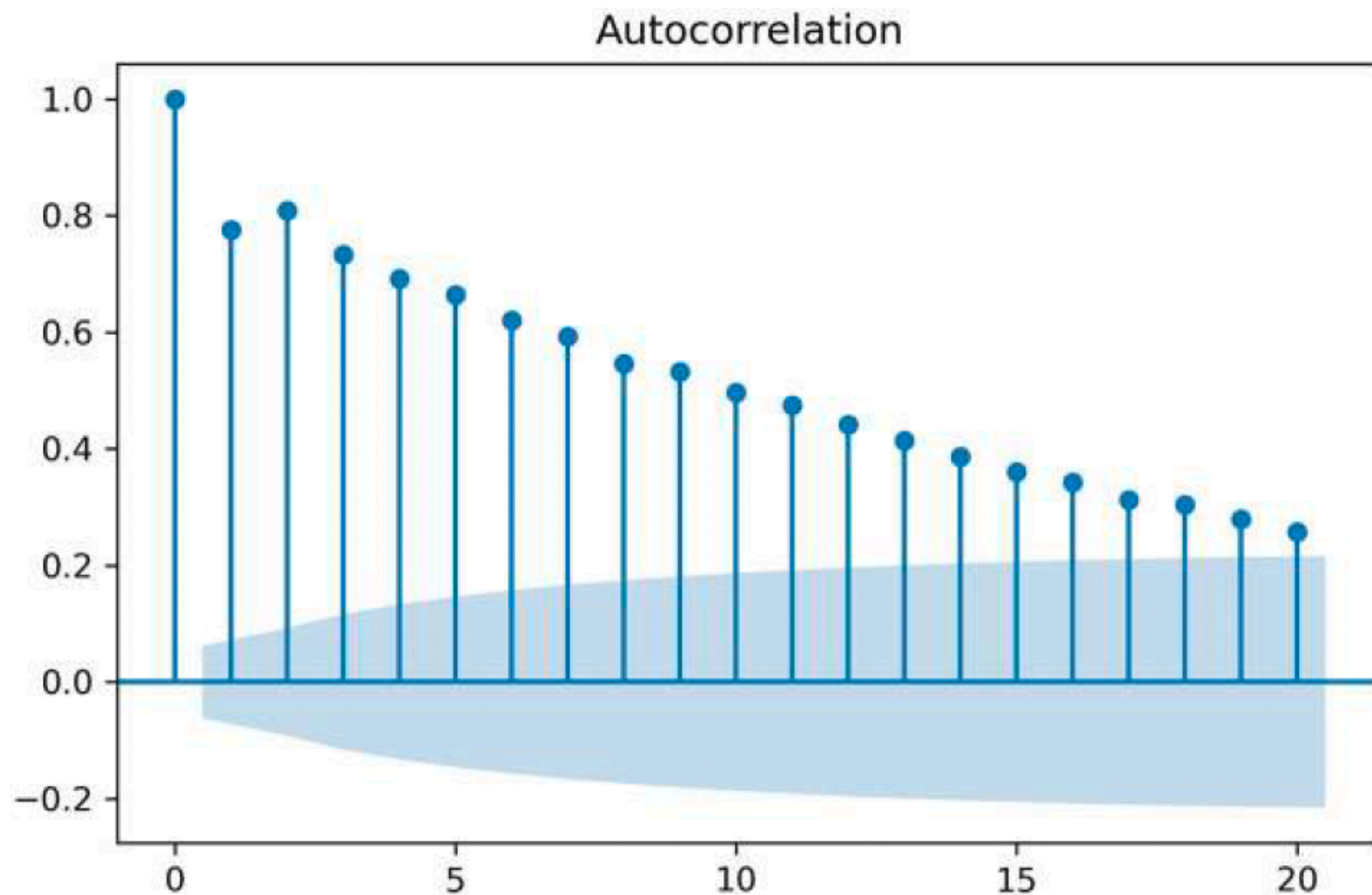
```
import numpy as np  
foot_traffic_diff = np.diff(df['foot_traffic'], n=1)
```

- Again check *stationarity*:

```
ADF_result = adfuller(df['foot_traffic_diff'])  
print(f'ADF Statistic: {ADF_result[0]}')  
print(f'p-value: {ADF_result[1]}')
```

This gives an ADF statistic of -5.27 and a p-value very small. So, the new series is stationary.

Implementation (Autocorrelation)

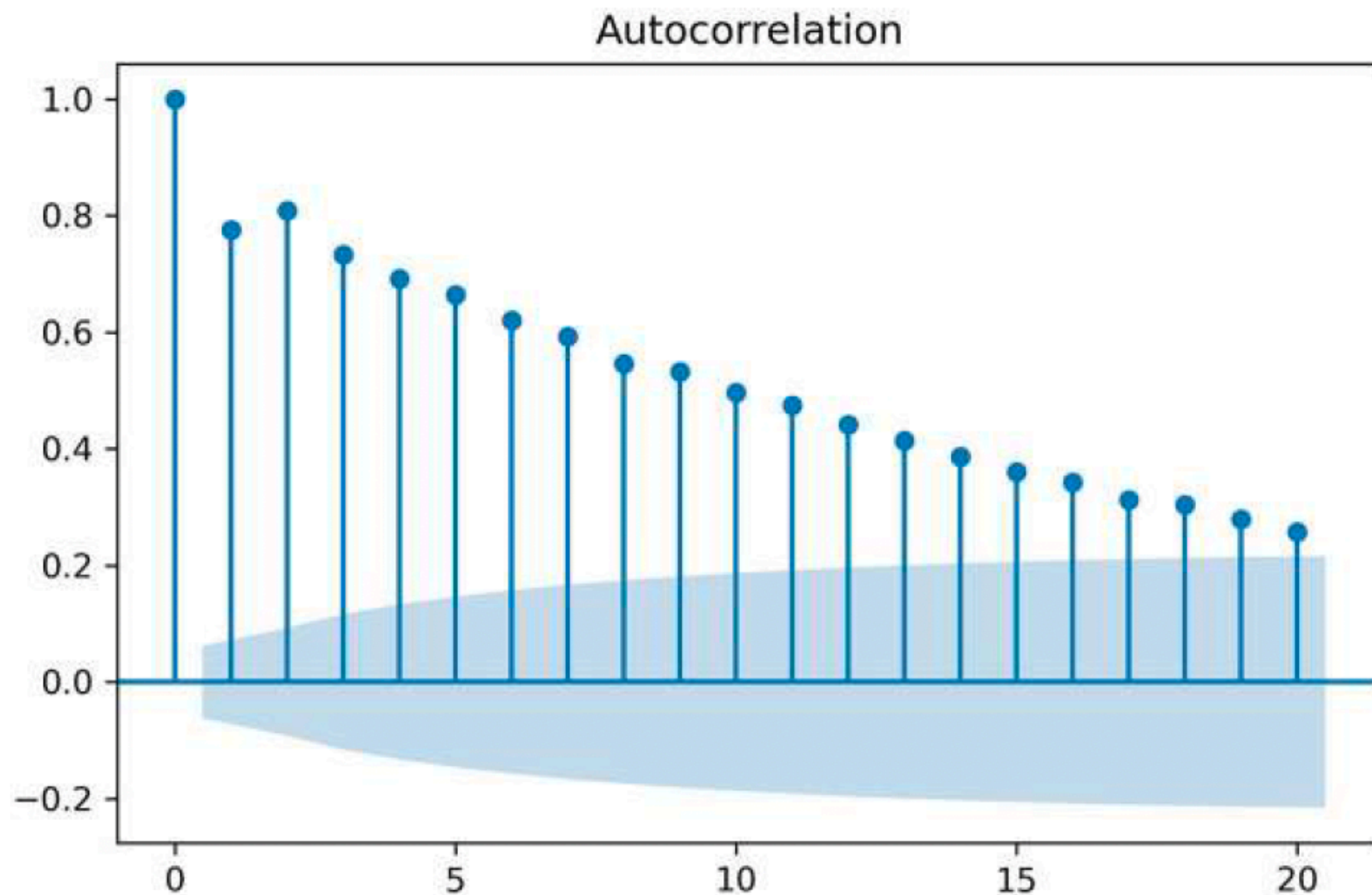


- The next step is to check whether the series is *autocorrelated*. Use the **plot_acf** function from the **statsmodels** library:

from statsmodels.graphics.tsaplots import plot_acf

plot_acf(foot_traffic_diff, lags=20)

Implementation (Autocorrelation)



- The next step is to check whether the series is *autocorrelated*. Use the **plot_acf** function from the **statsmodels** library:

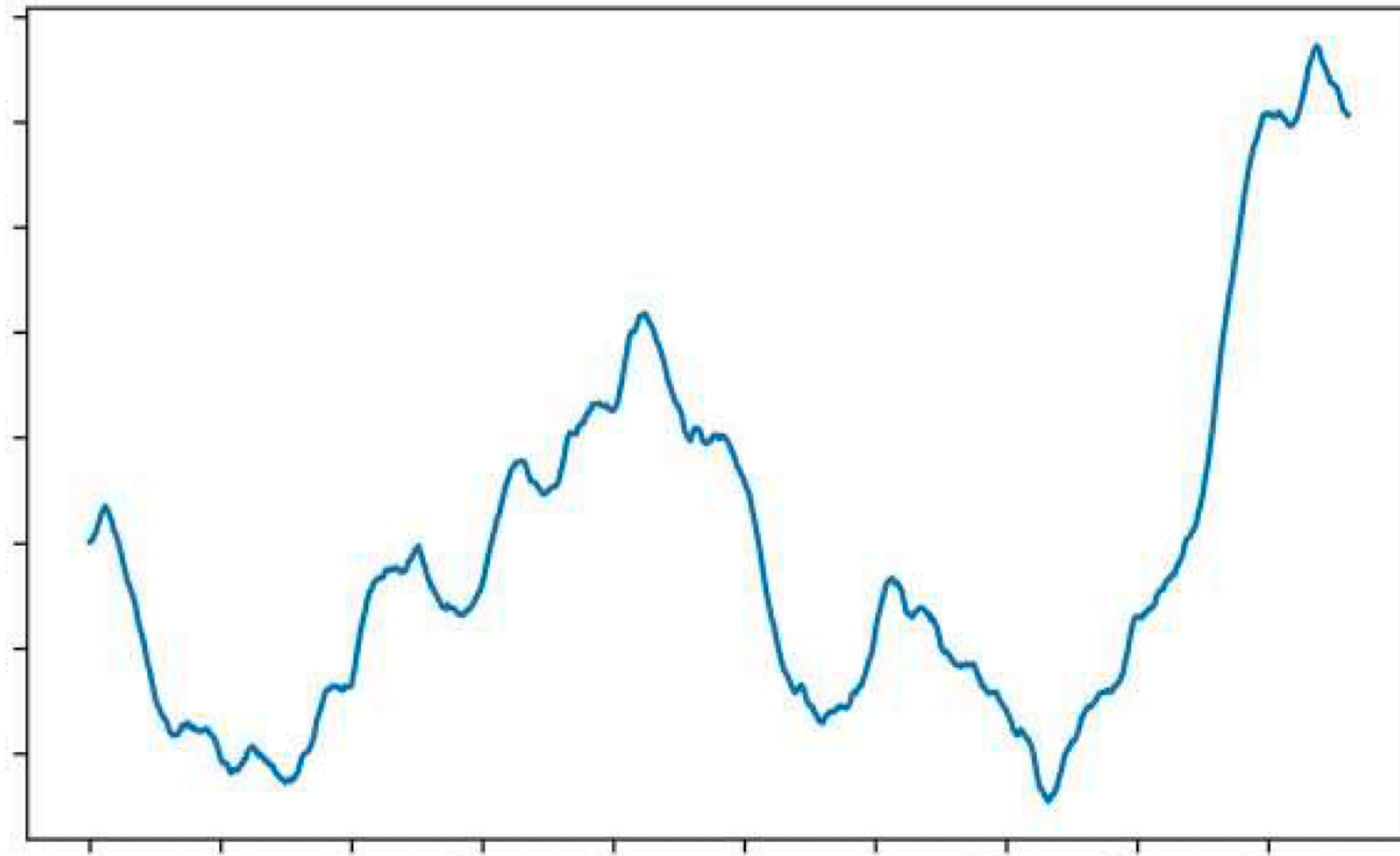
from statsmodels.graphics.tsaplots import plot_acf

plot_acf(foot_traffic_diff, lags=20)
- There are significant autocorrelation coefficients beyond lag 0. Therefore, we know that our process is **not a random walk**.
- The coefficients are decaying exponentially as the lag increases. Therefore, there is no lag at which the coefficients abruptly become non-significant. So, it is **not a moving average process** and that we are likely studying an **autoregressive process**.

04

Partial Autocorrelation Function (PACF)

Partial Autocorrelation Function (PACF)

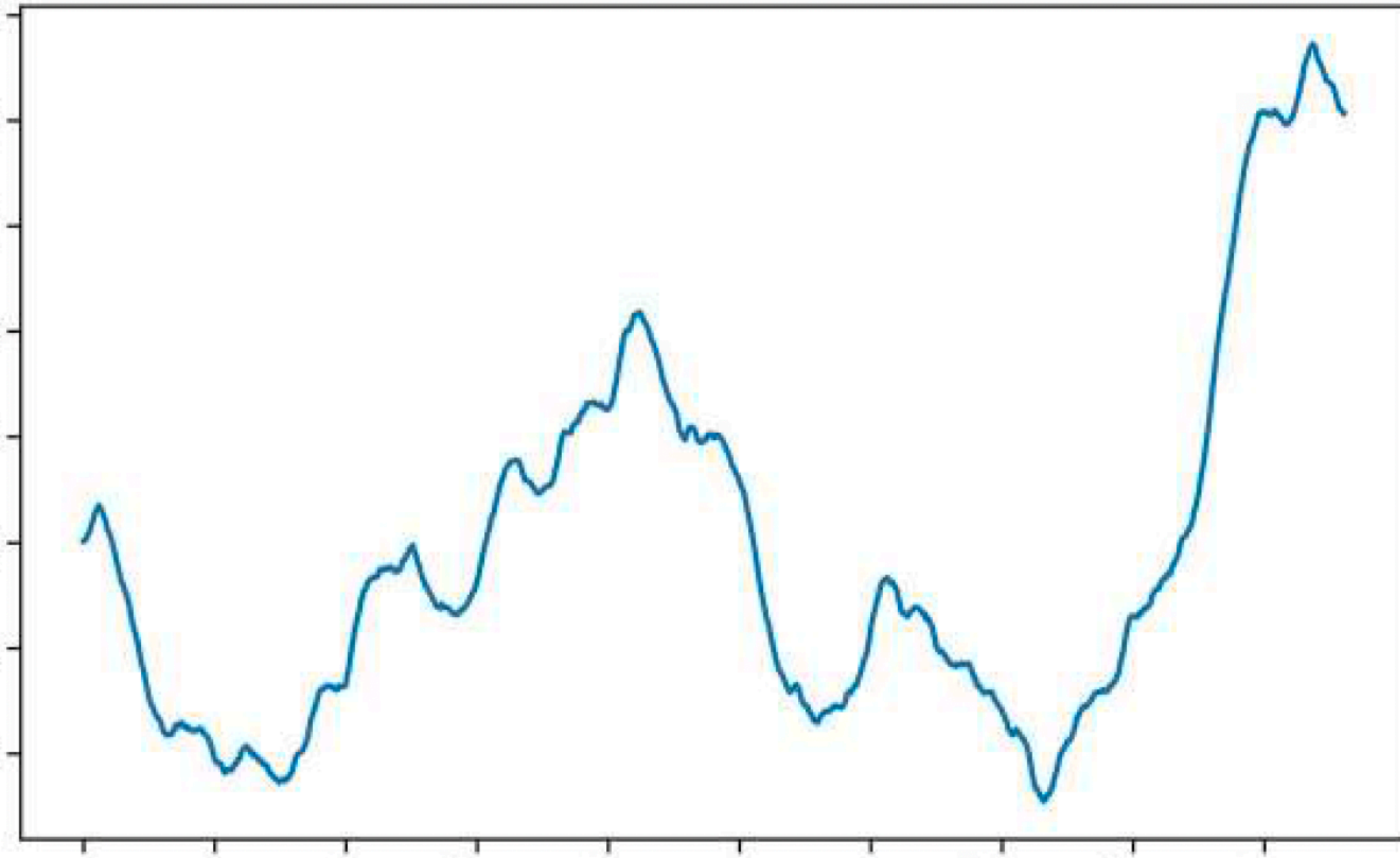


- **Autocorrelation** measures the linear relationship between lagged values of a time series.
- Consequently, the autocorrelation function measures how the correlation changes between **two values** as the lag is increased.
- Suppose we have the following AR(2) process:

$$y_t = 0.33y_{t-1} + 0.50y_{t-2}$$

We cannot measure the true impact of y_{t-2} on y_t , since there also is y_{t-1} which impacts both y_{t-2} and y_t .

Partial Autocorrelation Function (PACF)



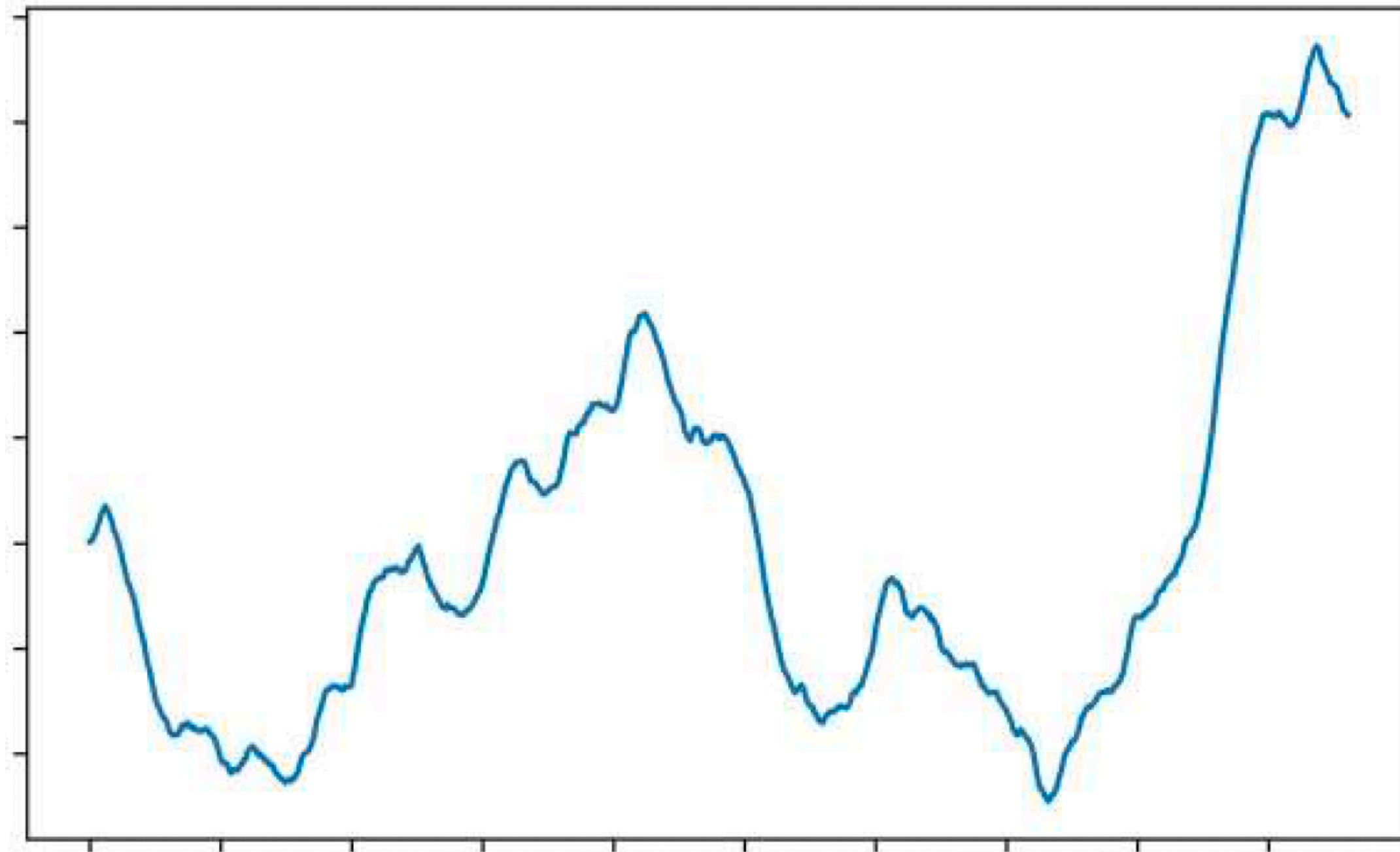
- Suppose we have the following AR(2) process:

$$y_t = 0.33y_{t-1} + 0.50y_{t-2}$$

We cannot measure the true impact of y_{t-2} on y_t , since there also is y_{t-1} which impacts both y_{t-2} and y_t .

- **Partial autocorrelation** measures the correlation between lagged values in a time series when we remove the influence of correlated lagged values in between.
- We can plot the partial autocorrelation function to determine the order of a stationary $AR(p)$ process. The coefficients will be non-significant after lag p .

Partial Autocorrelation Function (PACF)



- Let us simulate the AR(2) process

$$y_t = 0.33y_{t-1} + 0.50y_{t-2}$$

- Use the **ArmaProcess** function from **statsmodels**:

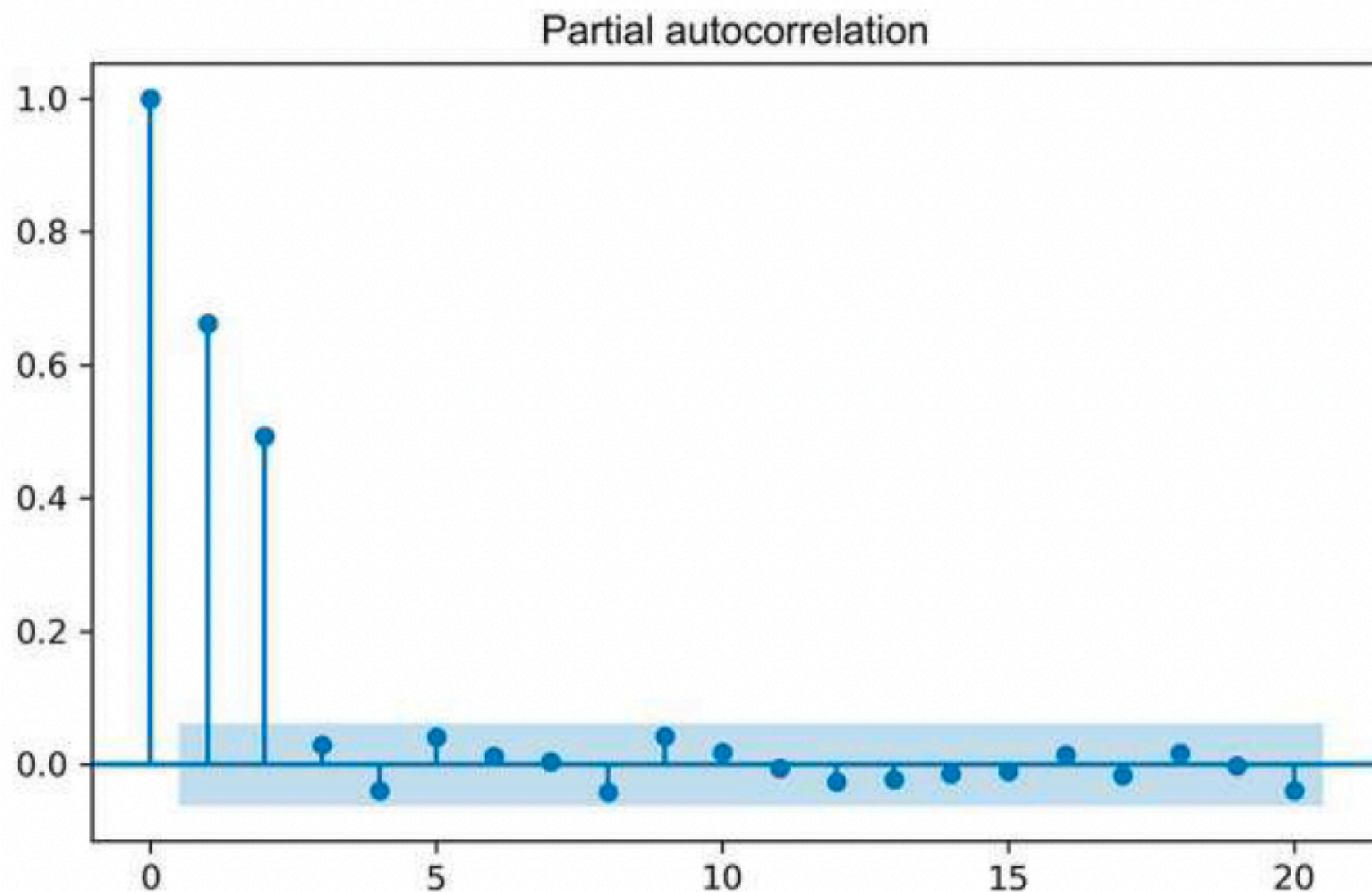
```
from statsmodels.tsa.arima_process import ArmaProcess
```

```
import numpy as np  
np.random.seed(42)
```

```
ma2 = np.array([1, 0, 0])  
ar2 = np.array([1, -0.33, -0.50])
```

```
AR2_process =  
ArmaProcess(ar2, ma2).generate_sample(nsample=1000)
```

Plot PACF (Simulated)

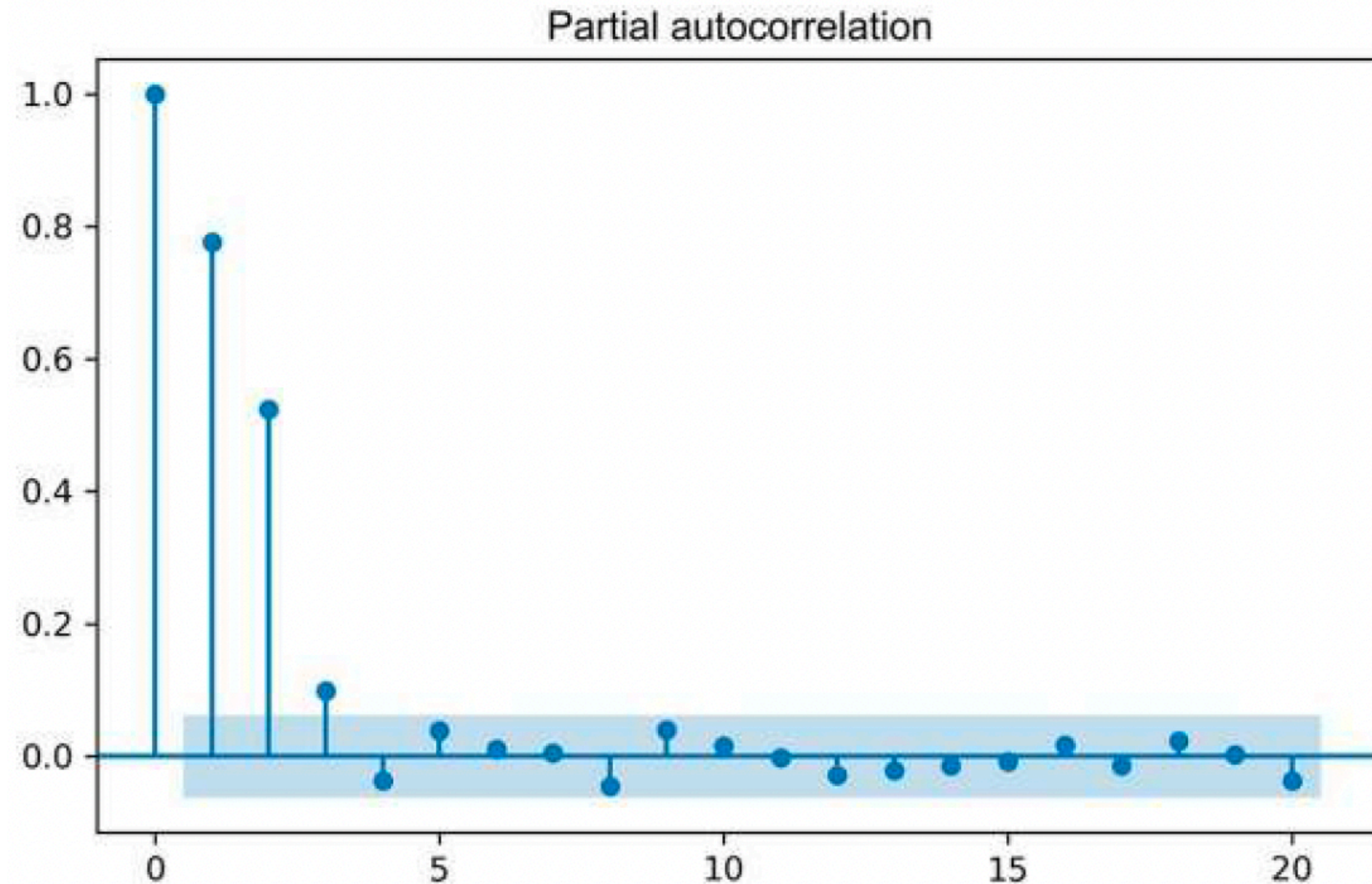


- The PACF plot to identify the order of a stationary $AR(p)$ process. The coefficients in the PACF plot will be significant up until lag p . Afterward, they should not be significantly different from 0.
- Use the **plot_pacf** function from **statsmodels**:

from statsmodels.graphics.tsaplots import plot_pacf

plot_pacf(AR2_process, lags=20)
- We can see that this is an $AR(2)$ process.

Plot PACF (Foot Traffic)



- Use the `plot_pacf` function from `statsmodels` for the foot traffic:

`plot_pacf(foot_traffic_diff, lags=20)`

- We can see that this is an AR(3) process.

Thank you!